

Notes of parallelism and concurrency in software

Dario Marcone

Jeudi 19 février 2026

Contents

1	Metrics	3
1.1	Iron Law	3
1.2	Amdahl's law	4
1.3	Different type of mean	5
2	Parallel computing	5
2.1	Expressing parallelism	5
2.2	Principles of parallel computing	6
3	Hardware cache coherence	8
3.1	Bus-based systems	8
3.2	Point-to-point systems	9
4	Optimizing software	11
4.1	Coherence misses	11
4.2	Work distribution and scheduling	14
5	SIMD and vector	14
5.1	AVX instructions	15
6	Message passing	17
6.1	MPI	18
6.2	MPI collectives	20
7	Memory consistency	22
7.1	Sequential consistency	22
7.2	Processor consistency	24
7.3	Weak consistency	24
7.4	Language level consistency	25
7.4.1	Java Memory Model	25
7.4.2	C++ 11 Memory Model	26
7.4.3	OpenMP Memory Model	27
8	Synchronization	27
8.1	Locks	28
8.1.1	TS	28
8.1.2	TTS	29
8.1.3	TS with exponential back-off	30
8.1.4	Load-locked and store-conditional	30
8.1.5	Coarse/fine-grained locking	31
9	Concurrency control	32
9.1	Deadlocks	32
9.1.1	Detection	33
9.2	Transactional memory	35
9.3	Point to point synchronization	36
10	Context switching	38
10.1	User threads	40
10.2	Coroutines	41
11	Microservices	44
11.1	RPC	46
11.1.1	gRPC	48
12	Multi-threaded processors	50
12.1	CGMT vs. FGMT	53
12.2	SMT	54

13 GPUs	55
13.1 GPU's hardware	56
14 CUDA	57

1 Metrics

1.1 Iron Law

$$\text{Processor Performance} = \frac{\text{Instructions}}{\text{Program}} \cdot \frac{\text{Cycles}}{\text{Instruction}} \cdot \frac{\text{Seconds}}{\text{Cycle}}.$$

- $\frac{\text{Instructions}}{\text{Program}}$:

The $\frac{\text{Instructions}}{\text{Program}}$ part represent the number of lines of code of your program, it is influenced by:

- The compiler: depends of how many machine instructions per line of code.
- The runtime system: depends of the language interpreted, the runtime overhead, etc

Remark

The runtime system is a program given with the programming language (ex. JVM for Java), that gives necessary services and mechanisms to the program during its execution, like memory allocation, etc. . .

- $\frac{\text{Cycles}}{\text{Instruction}}$:

The $\frac{\text{Cycles}}{\text{Instruction}}$ part represent the number of cycles by instructions, it is influenced by:

- The front-end cycles (instruction cache, branch predictor, etc. . .),
- The back-end cycles (data cache, ALU, FPU, etc. . .),
- The memory (latency, bandwidth).

Remark

Front-end part of the pipeline CPU is the instruction fetching, decoding and branch prediction part, and the back-end is the computation, memory access and data storage part.

We have to be careful! The cycles per instruction (CPI) is a good metric for uniprocessor, but not necessarily for multi processor. In multiprocessors, some core may have to wait for others, to have a specific information or whatever, so when the core wait, it will just do some spinning threads which is very fast $\implies$ a low CPI! But in fact, you don't execute the program particularly faster.

It is better to use some metric for “useful work”, like:

- Transactions complete per second,
- Request services per second.

- $\frac{\text{Seconds}}{\text{Cycle}}$:

The $\frac{\text{Seconds}}{\text{Cycle}}$ part is the clock frequency.

Vocabulary parenthesis:

- **Latency:** time it takes for an event to complete
- **Bandwidth:** how many events per unit time

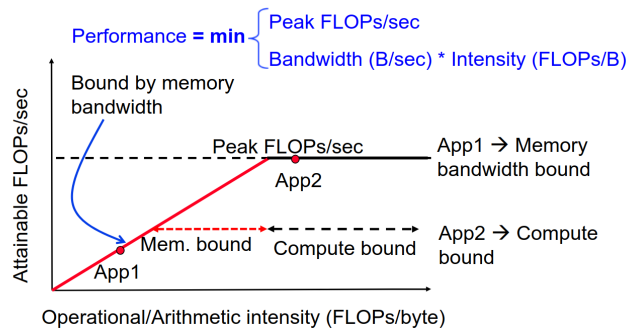
Reducing latency increases bandwidth but not vice versa.

1.2 Amdahl's law

The amdahl's law shows that we have to improve efficiency of what take the most of the time in our code execution.

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}.$$

Roofline Model



- The x-axis correspond to the number of data used for computation: if it is low, then it means you load/write a lot of data but you don't make computation with them. You are limited by the speed of loading/writing of/in the memory, not by the CPU calculation speed $\Rightarrow$ memory bound.
- The y-axis correspond to the number of FLOPs that you can reach on the CPU, if you are doing a lot of computation by byte, then the CPU reach its maximum calculation speed $\Rightarrow$ peak FLOPs/sec, independent of memory.
- We can know by what we are limited by taking the minimum of

$$\text{Performance} = \begin{cases} \text{Peak FLOPs/sec} \\ \text{Bandwidth} \cdot \text{Intensity} \end{cases}$$

It means that

if $B \cdot \text{Intensity} < \text{Peak FLOPs/sec} \Rightarrow$ we are limited by memory,

if $B \cdot \text{Intensity} \geq \text{Peak FLOPs/sec} \Rightarrow$ we are limited by CPU.

Parallel efficiency

$$\text{Parallel efficiency} = \frac{\text{Speedup}}{\# \text{ of cores}}.$$

Vocabulary parenthesis:

- **Speedup(n):** performance improvement of n cores over 1 core,
- **Costup(n):** cost increase of n cores over 1 core.

1.3 Different type of mean

- Arithmetic mean:

$$\frac{1}{n} \sum_{i=1}^n \text{Time}_i,$$

- Harmonic mean:

$$\frac{n}{\sum_{i=1}^n \frac{1}{\text{Rate}_i}},$$

- Geometric mean:

$$\text{Relative Rate} = \frac{\text{Rate}}{\text{Rate}_{\text{ref}}} = \frac{\text{Time}_{\text{ref}}}{\text{Time}},$$

- Geometric mean:

$$\sqrt[n]{\prod_{i=1}^n \text{Relative Rate}_i} = \frac{\sqrt[n]{\prod_{i=1}^n \text{Rate}_i}}{\text{Rate}_{\text{ref}}}.$$

2 Parallel computing

2.1 Expressing parallelism

There was different era of core computing, first the **simple core era**, we were executing one instruction per cycle.

Then in **pre-multicore era**, we had one BIG fancy core:

- A lot of budget on run a single instruction very fast,
- More transistors, so faster branch predictor, larger cache hierarchy, etc. . .

Finally, we are in the **multicore era**, more transistors, but in simple cores: each core is slower (around 25%), but there is (for example) two cores $\rightarrow 2 \cdot 0.75 = 1.5$.

But now, we have to parallelize our code, either it will run on only one of the cores. There is different way expressing parallelism as programmer:

- Use existing parallel code through libraries: Spiral, ScalAPACK,
- Writing in a parallel programming language: HPF, CAF, Java, Scala,
- Writing in compiler directives: OpenMP, Intel TBB,
- Writing using a threading library: MPI, PVM, Pthreads,
- Domain Specific Languages (DSLs): Spiral, CUDA.

PThreads:

```
1 void parallel_cbrt(float* v, float* result, int N) {
2     pthread_t thread_id;
3     my_args args;
4     args.N = N/2;
5     args.v = v;
6     args.result = result;
7     pthread_create(&thread_id, NULL, my_thread_start, &args)
8     my_cbrt(v + args.N, result + args.N, N - args.N);
9     pthread_join(thread_id, NULL);
10 }
```

OpenMP: We can parallelize simply when loop iterations are independent of each other.

```
1 void my_cbrt(float *v, float *result, int N) {
2     #pragma omp parallel for
3     for (int i = 0; i < N; ++i) {
4         float x = 1.f;
5         for (int j = 0; j < 10; ++j)
6             x = (1.f / 3.f) * (2.f*x + v[i] / (x*x));
7         result[i] = x;
8     }
9 }
```

Another idea of **multicore era**, is the **SIMD processing**: Single Instruction, Multiple Data. We add ALUs to increase our compute capability, the idea is to transform a loop where there is an instruction repeated on a lot of different data into a single instruction on a vector of a lot of data, then for a simple for loop:

```
1 for (int i = 0; i < 8; i++)
2     c[i] = a[i] + b[i];
```

We can compute this with one single operation of vector with the code:

```
1 __m256 va = _mm256_load_ps(a);
2 __m256 vb = _mm256_load_ps(b);
3 __m256 vc = _mm256_add_ps(va, vb);
```

`__m256` stand for the float type, the `_mm256` is the size of the vector, here 256 bits (8 float) and the load/add is the operation of the function.

GPUs are big SIMD processor!

2.2 Principles of parallel computing

- Finding enough parallelism,
- Division of work,
- Scaling,
- Communication and synchronization.

Finding enough parallelism: this is all the Amdahl's law part.

Division of work:

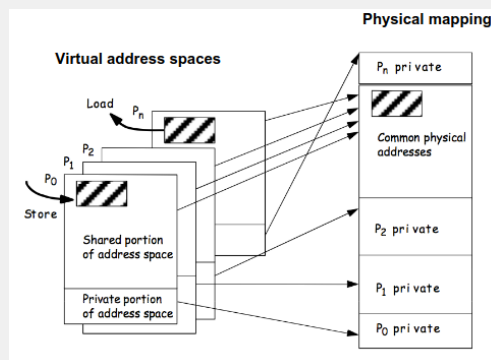
- Balance workload, give each parallel task the same rough amount of work.
- Reduce communication, computation is useful work, communication is overhead.
- Granularity, this is the ratio of computation to communication.

- Fine-grained parallelism: the idea is to split the work into very small tasks. There is not a lot of computation by thread but a lot of computation $\implies$ low granularity, but we need a fast communication favors: e.g. SIMD, GPU, ...
- Coarse-grained parallelism: the idea is to split the work into big tasks. We have less cache and TLB misses and less communication, we can tolerate, but its harder to divide the work into equal time part.

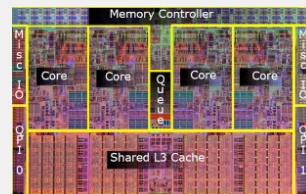
Communication & synchronization: We will see three different model of communication/synchronization:

- Message passing: the synchronization is at the I/O level, hadoop (e.g. datacenter).
 - There is compleety different address spaces, threads communicate via sending/receiving message, the synchronization is implicit in messages.
 - This is harder to program than shared memory, but more scalable.
 - Message passing is the most used communication system in datacenters.
- Shared memory: the synchronization is at the memory level, Scala (e.g. CPU).
 - Threads communicate by reading/writing to shared vars, that are like a big bulletin board (panneau d'affichage).
 - Any thread can read or write, communication is implicit in memory operations (no send/receive operations).
 - Non-uniform memory access (NUMA): all processors can access any memory location, but cost of memory access is different for different processors.
 - Implementation and some example:

Each thread has its own virtual address, but shared parts maps to the same physical location.



Any processor can directly reference any memory location (e.g. Intel Core i7 example (quad core)).



- Dataflow: the synchronization is at the processor level, TensorFlor (e.g. TPU).
 - In data-parallel computation, communication and synchronization are implicit, since each processor works on a different part of the vector and synchronizes (or joins) at the end.

3 Hardware cache coherence

A small reminder before starting: a cache contains usually the state (valid/invalid), the tag array and the data array that keeps cache blocks (a cache block == a cache line).

4Cs cache miss model:

- Compulsory: access data for the first time.
- Capacity: when the cache is not big enough.
- Conflict: when two addresses map to the same set and way of the cache evicting one of them.
- Coherence: when a block is removed due to coherence messages from another core.

Due to sharing data, incoherence appears between different caches. More specifically, when you're updating a value and reading a different copy. There is different ways to assure coherence we'll see two of them with their protocol.

3.1 Bus-based systems

In bus-based systems, we have to add independent "cache controllers" to each core where each controller: is a FSM, sees all remote processor actions, takes coherence actions depending on block's state and protocol.

MESI protocol: **M**odified, **E**xclusive, **S**hared and **I**nvalid states.

- Modified: this cache has updated the value and holds the authoritative copy.
- Exclusive: this cache has an exclusive copy.
- Shared: one or more caches hold the value.
- Invalid.

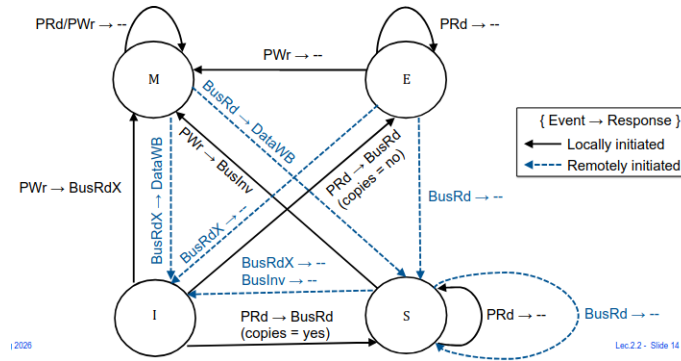
Core actions:

- PRd: processor attempts to read a line.
- PWr: processor attempts to write a line.

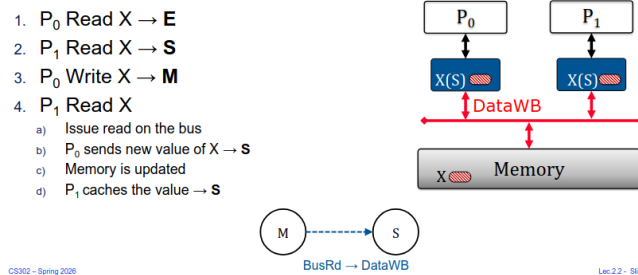
Remote actions:

- BusRd: obtain copy of a line with no intent to modify.
- BusRdX: obtain copy of a line with intent to modify.
- BusInv: get rid of other copies.
- DataWB: put updated value of a line on the bus.

MESI protocol state diagram:



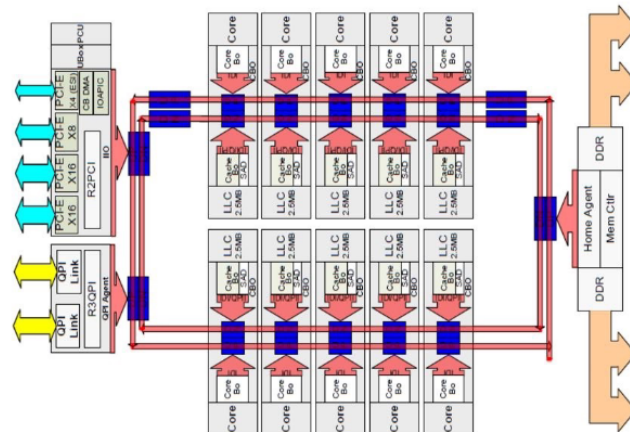
Example:



The main problem with bus-based systems is that the bus becomes a bottleneck: as the number of cores increases, the traffic on the bus grows, leading to contention and reduced performance. That's why we'll introduce the point-to-point network organization.

3.2 Point-to-point systems

In the point-to-point system, the message is more scalable and have an explicit source and destination. For example, Intel Skylake uses a ring:



Remark

In a ring (point-to-point) network, messages travel only along the path between sender and receiver, so links not on the path remain free, reducing overall traffic compared to a shared bus.

Now each core no longer sees all of the messages, so we need to add another protocol controller to the MESI protocol into the system.

Directory-based protocol:

In the directory-based protocol, we use a **coherence directory**. The directory tracks all caches that contain a particular block, then all messages leaving the core are first sent to the directory and it takes the appropriate actions, and respond to the original cache request. The directory serves as a per-block address or

Cache to directory messages:

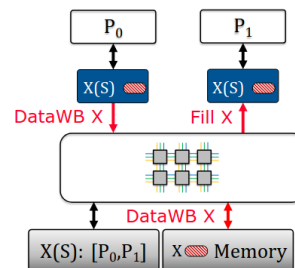
- GetRd: get copy to read,
- GetWr: get copy to modify,
- UpGrd: get rid of other copies,
- DataWB: write back a line,
- Ack: acknowledge message.

Directory to cache messages:

- DwnGrd: go from E/M $\rightarrow$ S,
- Inv: get rid of your copy,
- Fill: here is a copy of the line,
- Ack: acknowledge message.

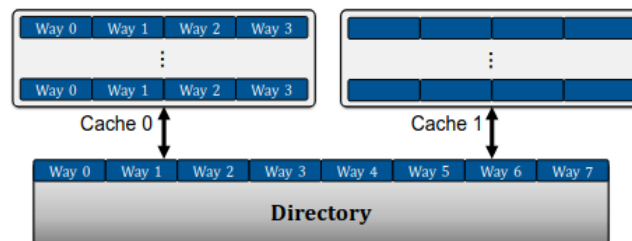
Example:

1. P_0 Read $X \rightarrow E$
2. P_1 Read $X \rightarrow S$
3. P_0 Write $X \rightarrow M$
4. P_1 Read X
 - a) Send request to directory
 - b) Directory sees P_0 has the most updated copy, and forwards P_1 's request $\rightarrow S$
 - c) P_0 sends P_1 the data $\rightarrow S$ and writes back to memory



The **directory entry** contain: 2 bits storing the MESI coherence state, 39 bits containing the cache tag $\log_2(\text{Cores})$ bits identifying the core associated with this entry.

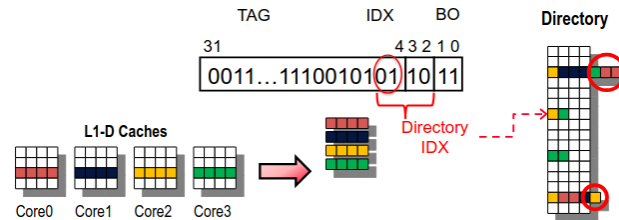
The size of directory is dependent of the number and the size of caches. If there is two processor with 4-ways associatives caches, then the directory must be 8-way associative:



But centralized directory can also be a source of contention, so we can distribute the directory across L2 cache banks, with each memory block having a home bank that tracks which L1 caches hold copies.

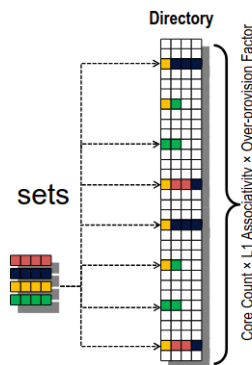
Access to the directory:

If we do the structure that we saw before for the directory, for four cores with 4-way caches, we need to have a 16-way associative cache! This is bad for latency and power. To allow this is the **sparse directory**, we can split the cache and take more tag bits to index.



Now the problem is that we can have conflict easily! The solution is to make a bigger directory and use more tag bits to index.

With this structure, we'll have low conflict rate, but a large area overhead...



4 Optimizing software

4.1 Coherence misses

Coherence adds extra misses when two cores access the same cache line, then there is two types of coherence misses:

- **True sharing:** the producer/consumer communication when processors read and write the same variable.
- **False sharing:** processors updating different data which are placed in the same cache block.

True sharing, histogram example:

We have an input text and we want to count the occurrences of each different characters in the text. We'll use OpenMP to divide the work across threads.

```

1 int histogram[N_BUCKETS];
2 int in_data[INPUT_SIZE];
3
4 #pragma omp parallel for shared (in_data, histogram)
5 for(int i = 0; i < INPUT_SIZE;i++) {
6     unsigned bucket = calc_hist(in_data[i]);
7
8     #pragma omp critical
9     histogram[bucket]++;
10 }

```

Then, there is true sharing when processors modify the histogram `histogram['a']++;`. To avoid this, we have to preserve correct value. Then we can

```

1 int histogram[N_BUCKETS];
2 int tmp_hist[N_THREADS][N_BUCKETS];
3 int in_data[INPUT_SIZE];
4
5 // Repartition loop
6 #pragma omp parallel for shared (in_data, tmp_hist)
7 for(int i = 0; i < INPUT_SIZE;i++) {
8     int tid = omp_get_thread_num();
9     unsigned bucket = calc_hist(in_data[i]);
10    tmp_hist[tid][bucket]++;
11 }
12
13 // Merge loop
14 #pragma omp parallel shared (tmp_hist, histogram)
15 for(int i = 0; i < N_BUCKETS; i++) {
16     int tid = omp_get_thread_num();
17     #pragma omp critical
18     histogram[i] += tmp_hist[tid][i];
19 }

```

Remark

In the code, `shared (x, y)` tells OpenMP that all threads should see and access the same variable in memory (shared memory communication).

In the first code, we have `INPUT_SIZE` times the critical section and the read/write to the histogram. In the second one, we have `N_BUCKETS` times the number of threads, then if the size of the input is way bigger than the number of buckets times the number of threads, we eliminate a lot of critical sections and true sharing.

Here is just an alternative of the merge loop that eliminate all critical sections, but every thread accesses a fraction of everyone's histogram.

```

1 // Merge loop
2 #pragma omp parallel for shared (tmp_hist, histogram)
3 for(int i = 0; i < N_BUCKETS; i++) {
4     int tid;
5     for (tid = 0; tid < omp_get_num_threads(); tid++) {
6         histogram[i] += tmp_hist[tid][i];
7     }
8 }

```

False sharing, odd numbers example:

```

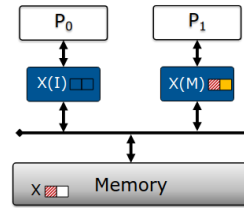
1 int counters[NTHREADS];
2 int in_data[INPUT_SIZE];
3
4 #pragma omp parallel for shared (in_data, counters)
5 for(int i = 0; i < INPUT_SIZE;i++) {
6     int tid = omp_get_thread_num();
7
8     if(in_data[i] % 2) {
9         counters[tid]++;
10    }
11 }

```

Nowadays, cache line are 64 byte in general, then it can store the four `int` on the same cache line. So everytime a core update its counter value, it invalidate every other cache line, this is false sharing!

1. P_0 Read $X_0 \rightarrow E$
2. P_1 Read $X_1 \rightarrow S$
3. P_0 Write $X_0 \rightarrow M$
4. P_1 Write $X_1 \rightarrow M$

- ◆ Block A continues bouncing between caches!
- ◆ Excess invalidations, and memory updates



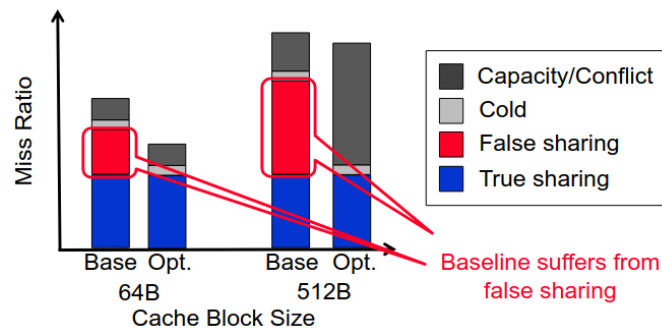
The solution here is to add padding, so the `int` are not on the same cache block anymore (e.g. 64 byte cache block line, a `int` is 4 byte, then we add a padding of 60 bytes).

```

1 typedef struct {
2     unsigned _c;
3     unsigned _padding[ BLK_SIZE/ UNSIG_SIZE - 1];
4 } PaddedCounter;
5 PaddedCounter counters[NTHREADS];
6
7 #pragma omp parallel for
8 {
9     ...
10    counters[tid]._c++;
11 }

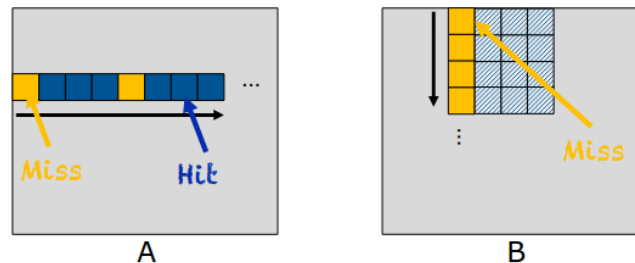
```

We saw that eliminate false sharing take a lot of space, so the capacity and conflict misses increases. Then, the bigger the cache is, the less the padding is efficient to reduce global misses.



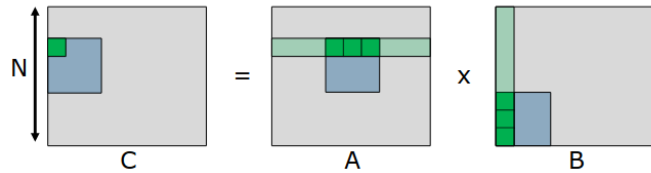
Block locality:

The locality of caches can be a problem in a program that, for example, multiply matrices, because we read the first matrix vertically and the second one horizontally.



Arrays are too large to fit in caches, then if we want to calculate $A \cdot B = C$, for every element of C if A and B are of size $N \times N$, we'll have $1 + N \left\lceil \frac{4}{\text{cache line size}} \right\rceil + N$ misses! Then for the whole calculation, we obtain $(N^3 (1 + c) + N^2)$ which is $O(N^3)$ misses (where $c = \frac{\text{element size}}{\text{cache line size}}$).

To avoid this, we can do blocking.
Then, for the whole array, we have to create $\left(\frac{N}{n}\right)^2$ blocks and they have $2 \cdot \left\lceil \frac{4 \cdot n}{\text{cache line size}} \right\rceil$ misses, so finally we have $O(N^2)$ misses.



4.2 Work distribution and scheduling

OpenMP is a join/fork model where fork creates a team of parallel threads and join, synchronize and terminate threads at the end of parallel region. But fork and join comes with an overhead.

For a code like this, if the loop has too few iterations, the fork and join overhead is bigger than the savings made by the parallel execution. There is no need to parallelize the loop.

```
1 #pragma omp parallel for
2 for(i = 0; i < n; i++)
```

We can use the `if` clause to resolve this: `#pragma omp parallel for if(n > 5000)`.

5 SIMD and vector

We already introduce SIMD in the **section 2.1**. In the basic scalar pipeline, execution is in one simple ALU, and this code

```
1 void daxpy(int n, double a, double* x, double* y){
2     for(int i = 0; 0 < n; i++){
3         y[i] = a * x[i] + y[i];
4     }
5 }
```

would look like this:

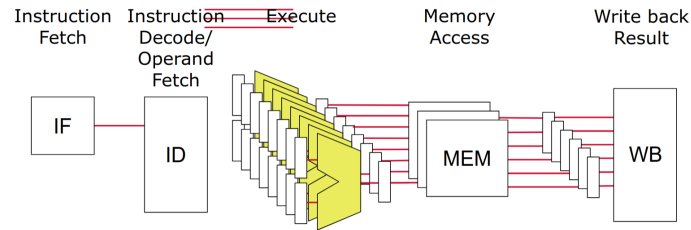
```
1 loop:
2     lw r1, 0(r2) //load x[i]
3     lw r6, 0(r3) //load y[i]
4     mul r1, r1, r4 //a*x[i]
5     add r6, r1, r6 //... + y[i]
6     sw r6, 0(r3) //store y[i]
7     add r2, r2, 8
8     add r3, r3, 8
9     bne r2, r5, loop
```

If we are doing 1024 iterations, we'll have $8 \cdot 1024 = 8192$ instructions.

We can count the number of misses too, a size of double is 8B, a cache line 64B $\Rightarrow$ 8 elements per block. We have two loads by loop, then we know that we have $\frac{2 \cdot 1024}{8} = 256$ misses.

If we want to optimize this by having a out-of-order processor and a re-order buffer (c.f. computer architecture), knowing that a ROB is of size ≈ 500 instructions, it is only 6% of instructions that can be in the ROB. We can't augment the ROB size because it adds too much overhead $\Rightarrow$ out-of-order processors are not a very good solution.

Let's introduce vector pipeline:



The vector processors:

- Reduces instruction bloat,
- Simplifies dependency checking,
- Potentially reduce cache misses,
- Reduces control hazards.

Examples of vector instructions:

- `mul.v v1, v2, v1` does the dot product $v1 \cdot v2$.
- `mul.sv v1, r2, v1` multiplies scalar `r1` to all elements of `v1`.
- `lw.v v1, 0(r1)` loads vector `v1` from address `r1`.
- `sw.v v1, 0(r1)` stores vector `v1` at address `r1`.

Then our compiler code would look like this:

```

1 loop:
2   lw.v v1, 0(r2) ; load x[i]
3   lw.v v2, 0(r3) ; load y[i]
4   mul.sv v1, r4, v1 ; a*x[i]
5   add.v v1, v1, v2 ;   + y[i]
6   sw.v v1, 0(r3) ; store y[i]
7   add r2, r2, 512 ; 64*8
8   add r3, r3, 512
9   bne r2, r5, loop

```

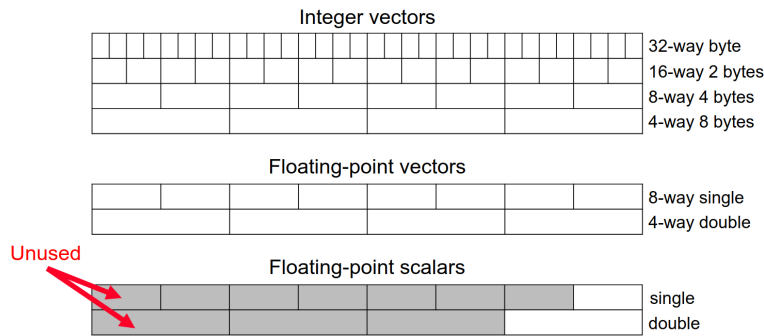
Now for 1024 iterations of our loop, we only have $8 \cdot \left(\frac{1024}{64}\right) = 128$ instructions. For misses, the vector load brings 64 elements at a time $\implies 8$ cache lines $\implies 2 \cdot \left(\frac{1024}{64}\right) = 32$.

To apply SIMD we need a specific hardware:

- Vector registers to store multiple elements of a vector in a single register,
- Replicated functional units inside the pipeline of a core (multiple lane/ALUs),
- Support for decoding new instructions (SIMD instructions).

5.1 AVX instructions

AVX instructions are for 256-bit operations on x86 architecture.



Let's interpretate an AVX instructions e.g. `vaddpd`:

- `v` means that is an AVX instructions,
- `add` indicate the operation,
- `p` means packed, it can also be `s`, then it would be scalar (you can see upper),
- `d` stand for double-precision, can be `s` for single-precision, etc...

To use AVX instructions there is multiple way, the simplest is to auto-compile the code by using `-O3` when compiling C code. But there is two main problems with the auto-compile solution. Aliasing and alignment.

Aliasing:

Let's take again our `daxpy` code, the compiler can't say at 100% that pointers are independent, then it won't vectorize. To solve this we have to say to the compiler that pointers don't overlap by changing the signature into

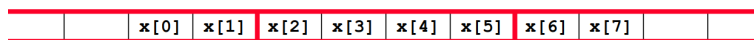
```
1 void daxpy(int n, double a, double* restrict x,
    double* restrict y);
```

Alignement:

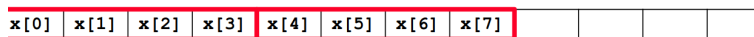
Modern DRAM accesses are per four bytes and for better performances, accesses to vector elements should be aligned.

for example, to load an 8-element 32-bit vector:

Case 1: Need three loads to load a misaligned vector



Case 2: Need two loads to load an aligned vector



To solve this, you can use `aligned_alloc` in C when allocating arrays and `_attribute__((aligned(32)))` for static arrays.

To summarize, the compiler cannot vectorize if:

- There is loop with unknown number of iterations at runtime,
- There is if-else statements or diverging condition inside loop,
- There is dependence between loop iterations,
- There is function calls in the loop body.

Another way to use AVX instructions is to optimize the code manually with “intrinsics”. Intrinsics are compiler-provided functions that map directly to specific CPU instructions.

Different intrinsics data types

Type	Meaning
<code>__m256</code>	8x single-precision floats
<code>__m256d</code>	4x double-precision floats
<code>__m256i</code>	32x 8-bit / 16x 16-bit / 8x 32-bit / 4x 64-bit integers
<code>__m128</code>	4x single-precision floats
<code>__m128d</code>	2x double-precision floats

Let’s interpretate an AVX intrinsics function e.g.

`__mm256_op_suffix(type param1, type param2, type param3):`

- `mm256` means that we are working on the 256-bit registers,
- `op` is the operation to do, e.g. add, sub, load, etc...
- `suffix` is the type of the returned vector, e.g. ps, pd, epi32, epu64, etc...

With this, our **daxpy** code would become

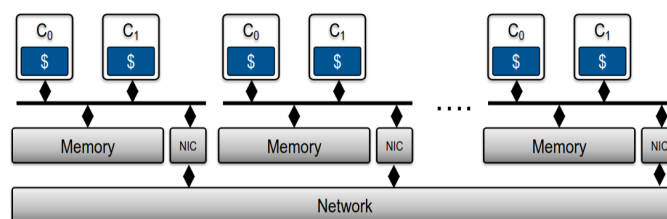
```

1  __m256d vecA = _mm256_set_pd(a, a, a, a);
2
3  for(int i = 0; i < n; i+=4) {
4      __m256d vecX = _mm256_load_pd(&x[i]); //Load x[i]...
5      __m256d vecY = _mm256_load_pd(&y[i]); //Load y[i]...
6      __m256d vecZ = _mm256_mul_pd(vecA, vecX); // a*x
7
8      vecY = _mm256_add_pd(vecZ, vecY); // ... + y
9
10     // Store result back to y
11     _mm256_store_pd(&y[i], vecY);
12 }
```

AVX intrinsics C code optimize a lot normal code because cache misses are optimized too.

6 Message passing

Message passing is used when the memory is distributed, it happens and it’s very useful for supercomputer or datacenters. So we have to use message passing when we have a particular hardware where: each node has its own memory, there is a network interface card (NIC) that connects nodes to a network.



Advantages of message passing are:

- The cost scalability: cache-coherent shared memory becomes expensive quite fast and distributed memory without cache coherence is common.
- The transparency: the communication is explicit (send/receive messages).
- The fault tolerance: if a process fail it doesn’t crash the entire system.

Disadvantages of message passing are:

- There is a higher communication overhead: syscalls, user-level library, etc ...
- It increases the code complexity.
- It is difficult to overlap with computation: while the message is travelling its hard to compute at the same time.

6.1 MPI

To apply message passing we use **MPI (Message Passing Interface)**. The model of MPI is a **SPMD (Single Program Multiple Data)** model: each process has a copy of the same program and they all run at their own rate and can take different paths through the code.

A simple code using MPI looks like this:

```
1 #include "mpi.h" //include the MPI libraries
2 #include <stdio.h>
3 int main(int argc, char *argv[])
4 {
5     MPI_Init(&argc, &argv); //Initializes MPI env.
6     printf("Hello, world!\n");
7     MPI_Finalize(); //End of MPI section
8     return 0;
9 }
```

MPI processes can be collected into groups and MPI identifies groups by their context. **MPI communicators** are combination of a group and its context. Groups can sometimes overlap.

Some facts on communicators:

On starting an MPI program there is one predefined communicator called `MPI_COMM_WORLD` that contains the group of all processes.

A process is identified by a unique number within each communicator, its called the rank of a process, and the same process can have different rank in different communicators when groups overlap.

The utility of communicators is to minimize unnecessary communications, but simple programs usually only use `MPI_COMM_WORLD`.

If we want to customize our “Hello world”:

```
1 #include "mpi.h"
2 #include <stdio.h>
3 int main( int argc, char *argv[] )
4 {
5     int rank, size;
6     MPI_Init( &argc, &argv );
7     //Get the rank of the process
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     //Get the # of the process
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11    printf("I am process %d of %d.\n", rank, size);
12    MPI_Finalize();
13    return 0;
14 }
```

In MPI both process must take action to communicate: one have to send and the other have to receive.

Send: `MPI_Send` is a blocking function call, it returns when message has been delivered to the system but it doesn't imply that the message has been received by the target process. There exist variant with different effects:

- `MPI_Bsend`: it returns when the message is buffered in an application buffer (doesn't wait at all before returning),
- `MPI_Ssend`: it returns only when receiver has started to receive the message,
- `MPI_Rsend`: assuming receiver is ready (is waiting), message is sent asap.

Receive: `MPI_Recv` is also a blocking function call, its source can be `MPI_ANY_SOURCE` to accept data from any sender and the tag can be `MPI_ANY_TAG` to accept messages with any tag. The status contains further information but we can ignore with `MPI_STATUS_IGNORE`.

Communicating arrays:

There is different approach to send arrays with MPI, the first one would be to send elements one by one, or to send the entire array at once.

```
1 int A[1000];
2 for(int i = 0; i < 1000; i++) {
3     MPI_Send(&A[i], 1, MPI_INT, rank, tag,
4             MPI_COMM_WORLD);
5 }
```

Or,

```
1 int A[1000];
2 MPI_Send(A, 1000, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

A hybrid approach is often the best (chunks of 100 elements), the first approach creates too much overhead and the second one solicit the memory and spam the cache if arrays are too big. For short messages, latency dominates transfer time, but for long messages this is bandwidth.

The **critical message size** is the amount of buffer that exists in the system, you can find it by computing

$$\text{latency} \cdot \text{bandwidth}.$$

Ping-pong between two processes example:

```
1 int ping_pong_count = 0;
2 int partner_rank = (rank + 1) % 2;
3 while (ping_pong_count < PING_PONG_LIMIT) {
4     if (rank == ping_pong_count % 2) {
5         // Increment the ping pong count before you
6         // send it
7         ping_pong_count++;
8         MPI_Send(&ping_pong_count, 1, MPI_INT,
9                 partner_rank, 0, MPI_COMM_WORLD);
10        printf("%d sent and incremented %d to %d\n",
11               rank, ping_pong_count, partner_rank);
12    } else {
13        MPI_Recv(&ping_pong_count, 1, MPI_INT,
14                 partner_rank, 0, MPI_COMM_WORLD,
15                 MPI_STATUS_IGNORE);
16        printf("%d received %d from %d\n", rank,
17               ping_pong_count, partner_rank);
18    }
19 }
```

MPI functions are blocking functions, it means that sometimes deadlocks can occurs if the code isn't written correctly.

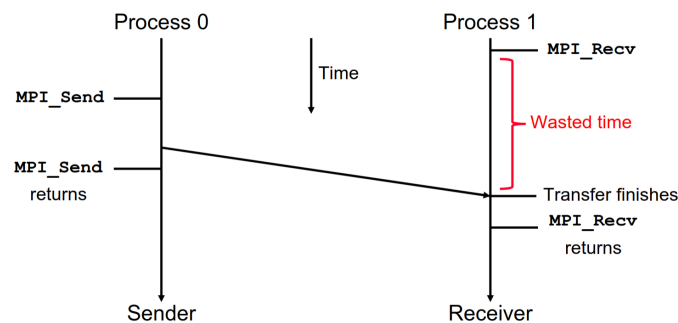
For example, in this code, the process 0 is stuck sending data, and process 1 is also stuck sending data. Neither can invoke `MPI_Recv` to the data.

```
1 if(rank == 0){
2     MPI_Send(...);
3     MPI_Recv(...);
4 } else if(rank == 1){
5     MPI_Send(...);
6     MPI_Recv(...);
7 }
```

Here, to fix this problem if order is known, we simply have to order functions correctly.

```
1 if(rank == 0){
2     MPI_Send(...);
3     MPI_Recv(...);
4 } else if(rank == 1){
5     MPI_Recv(...);
6     MPI_Send(...);
7 }
```

Due to blocking, we also have performance blocking:



To avoid this we can use **asynchronous communication**. Non-blocking function calls return immediately after invocation but they doesn't provide any guarantee on the status of the message, so the programmer have to check manually for completion of message transfer. It requires more programmer effort but allows to overlap communication with computation and increase the efficiency and performance of the program.

Asynchronous transfer can be used in this situation. Process does not wait before moving on to different data.

```
1 *buf1 = 3;
2 MPI_Isend(buf1, 1, MPI_INT, ...
3 *buf2 = 4;
```

But in this situation it's not possible. This is non-deterministic, receiver can get either 3 or 4.

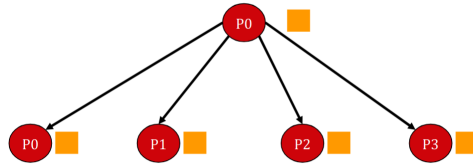
```
1 *buf1 = 3;
2 MPI_Isend(buf1, 1, MPI_INT, ...
3 *buf1 = 4;
```

6.2 MPI collectives

Some MPI instructions:

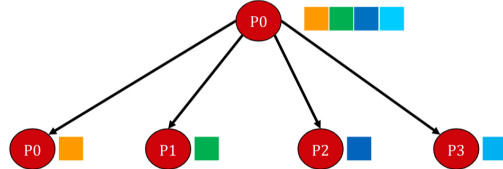
- **Broadcast:**

It sends the same data to all processes in the group in a time $O(\log_2(n))$.



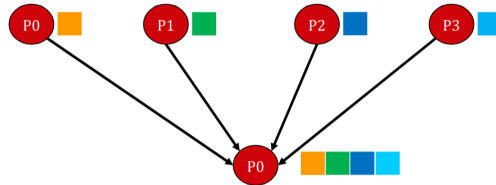
- **Scatter:**

It sends elements in an array to consecutive processes in a group, the data is transmitted in increasing order of process rank.



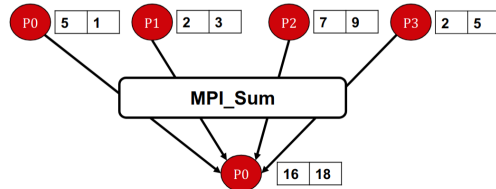
- **Gather:**

It gathers elements from consecutive processes in group into an array.



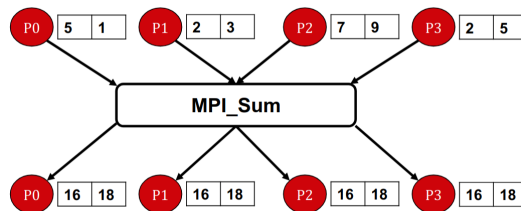
- **Reduce:**

It takes an array of input elements on each process and returns an array of output elements to the root process given a specific operation.



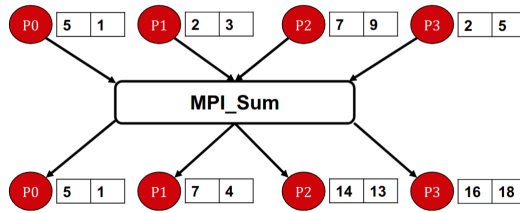
- **AllReduce:**

It's similar to reduce but results are distributed to all processes



- **Scan:**

Similar to AllReduce but with partial results



7 Memory consistency

Cache coherence simulate a unique shared memory, so we don't have to think about the location of different data. **Memory consistency** determine in which order instructions are simulated, because the processor can re-order instructions, we have to take care about memory consistency. It tells which re-ordering operations are acceptable or not. The memory consistency affects performance!!

To express this we can just look at this example

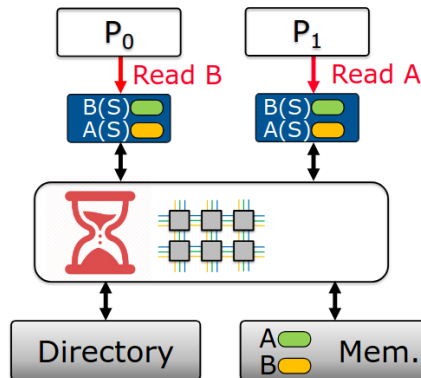
If we want to execute this two pieces of code, one in a processor, the other in another.

```
1 // A = r0 = 0
2 (S0) A = 1;
3 (L0) r0 = B;
4 print(r0);
```

```
1 // B = r1 = 0
2 (S1) B = 1;
3 (L1) r1 = A;
4 print(r1);
```

We can obtain (0,0) as result, but we don't expect that. Why we can obtain this? Because processors can execute this code like this

First, A and B are in shared state in both processors, then processors execute the stores simultaneously. While value are in the storing process, non-blocking caches proceed and the read hits. It returns (0,0).



To avoid this we must define a rigorous memory model to tell each layer what can and can't happen. We'll see different model.

7.1 Sequential consistency

The goal of **sequential consistency** is that the multiprocessor should behave like a multitasked single processor. The result of any execution is the same as if the operations of all processors (cores) were executed in some sequential order, and the operations of each individual processor (core) appear in this sequence in the order specified by its program.

The memory appears like it has a switch in front and like if each processor's memory accesses is executed atomically and in program order.

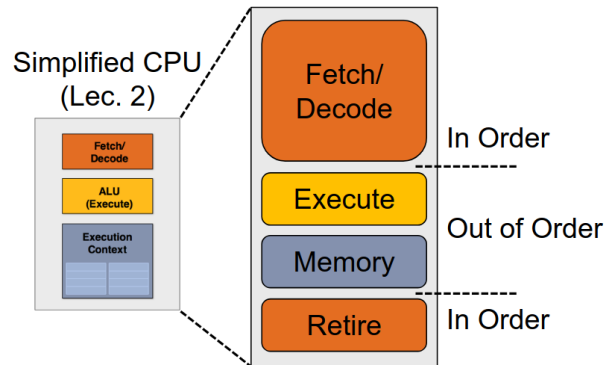
Remark

When we say that it appears atomic, it means that a write transitions directly from “not visible” to “fully visible,” with no observable intermediate state. Then, it ensures that a write is either visible, either invisible, not half-half.

To implement sequential consistency we have to ensure two things

1. Memory accesses happen in order.
2. Memory operations appear atomic.

To implement, we have to recall that the processor initially act like this

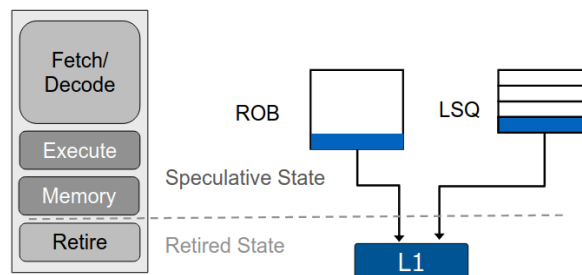


For the sequential consistency, we need to order memory instructions, using a **load-store queue (LSQ)** that will do the following things:

- Track the FIFO (first in first out) program order of loads and stores,
- Resolve addresses when they are ready,
- On a load, check for the **youngest** store to this address.

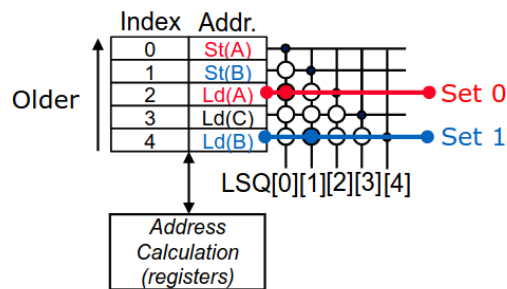
Load store queue functionality:

The LSQ resolves which load/store addresses overlap and hold store operations until they retire. It cannot write “speculative” values to caches, a speculative value is a out of order value. It means that the LSQ cannot exit an entry that is still in the ROB.



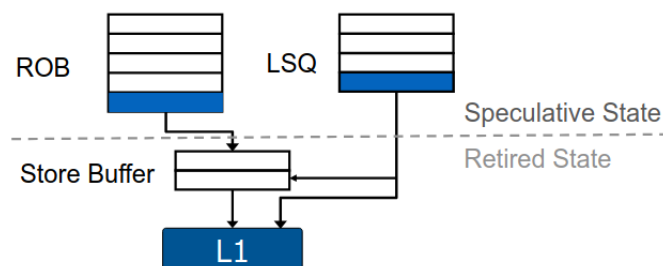
At address resolution of each load, we have to check if there is an older write in the LSQ at the same address. To do this, we use a matrix of comparators, that checks every entry against all older ones. In the example of loads after store, loads sets bits for all older stores they depend on, then the store resets its column when updating the cache (commit).

In this picture, each store have its columns, when the load address resolution happens, a 1 is set on the column of the store if this is the same address. Then, when the store quit the LSQ, its column is reset to 0.



The problem now is that we have to wait for store to complete! If this is a miss in L1/L2/LLC, it can takes 100+ cycles! The goal is that the processor continue while store pending.

The solution is to add a **store buffer (SB)**. It sits between core and L1 cache and holds committed stores.



In the LSQ, we have the address of stalled operations, why not using them?

We can't because it violates order or atomicity, but, in fact, we can use them by peeking. The idea is to peek at L1, to see if address is already there, if not, we fetch the block from lower level into L1. If the value is there, we **don't** load the value into the core. Fetching blocks into L1 from lower levels or other L1's doesn't impact order or atomicity, there is no values moved between the core and L1. We then can fetch as many blocks in parallel into L1 as needed.

7.2 Processor consistency

In processor consistency, reads in the LSQ can bypass writes in the SB.

If we have instructions in store buffer, then the differences between sequential consistency with peek and processor consistency will be

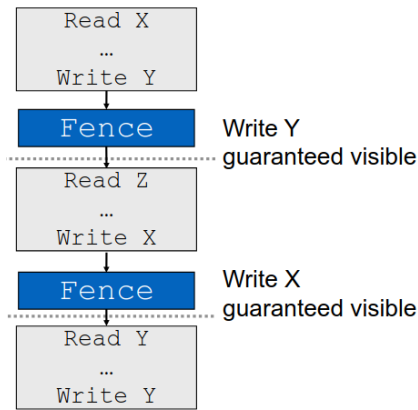
SC+Peek		PC	
Load Y	Peek	Store Z	Peek
Store A	In L1	Load Y	L1 miss
Load A	In L1	Store A	SB full

In sequential consistency with peeking, load A hits L1, but still had to wait, in processor consistency, load A can bypass stores in the store buffer.

7.3 Weak consistency

In **weak consistency** there is two sorts of memory operations, data operations or synchronization operations. Only synchronized one have any ordering, data operations have no order enforced among themselves.

Synchronization instructions are called **Fences**, it enforces program order for operations before/after.



7.4 Language level consistency

A **data race** is when two conflicting operations from different threads occur simultaneously. Operations conflict if they access the same location, and at least one is a write and these simultaneous operations are defined back-to-back accesses in any sequentially consistency.

If we check data races in our *first example* we can see that there is sequential consistency executions where read/write to A or B are back to back:

- $A = 1$
- $r_1 = B$
- $B = 1$
- $r_2 = A$

To make a data race free (DRF) code, we can do multiple things. Either we add add fences, by writing these lines: `_sync_synchronize();`
The new way is to declaring A, B as special variables (e.g. `atomic int`).

DRF implies sequential consistency:

Before we needed to adjust our programs depending on consistency model in th ISA. But with DRF even on hardware that uses PC or WC, our program sees it as SC, the programmer simply needs to obey the rules.

7.4.1 Java Memory Model

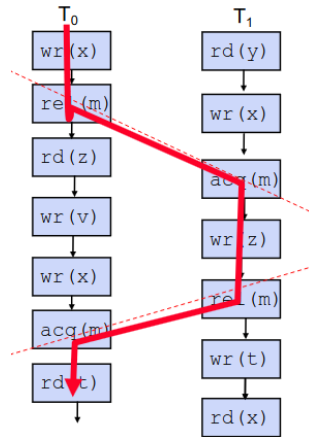
Memory order in Java is defined by “**happens-before**” relationship, these defined which re-ordering are possible in both the compiler and the JVM. Without the happens-before guarantee, the compiler/JVM can reorder instructions. From now, we’ll write happens-before as an arrow: $\rightarrow$.

There is five situations where $\rightarrow$ is guarantee by Java:

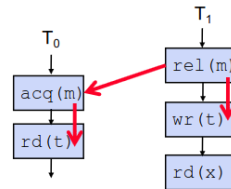
- Actions in the same thread $\rightarrow$ each other in program order.
- Unlock a monitor $\rightarrow$ all locking operations. It means that a thread cannot hat is already locked by another thread, it must wait until the other thread unlocks it.
- Writing to a volatile $\rightarrow$ all subsequent reads of that field.
- All actions in a thread $\rightarrow$ a `join()` on that thread.

- If $A_x \rightarrow A_y$, and $A_y \rightarrow A_z$, then $A_x \rightarrow A_z$.

If we can find two actions A_x such that A_x doesn't $\rightarrow A_y$, and A_y doesn't $\rightarrow A_x$, we have found a data race.



But sometimes, some actions may or may not happen before another. Then, $wr(t)$ is not guaranteed to happen before $rd(t)$, so there is a data race.



To eliminate data race we can just add `synchronized { }` blocks or `@volatile` variables.

Synchronized blocks versus volatile:

Synchronized blocks can only work with objects, the thread can wait while others are in synchronized blocks and they can wrap whole functions or critical sections.

On this code, another thread will either see both `a` and `x` to be updated neither `a` nor `x` to be updated

```
1 synchronized {
2     a = b + c;
3     x = foo ();
4 }
```

Volatile can be used with primitives (e.g. `int`), there is no synchronization/direct access and it prevents specific memory re-ordering.

If another thread sees `x` to be updated it must also see `a` to be updated. But reverse is not necessarily true.

```
1 @volatile int a, x;
2 a = b + c;
3 x = foo();
4 // can't reorder before
5 // wr(a)
```

7.4.2 C++ 11 Memory Model

All thread can read/write to any location in the program scope through references and pointers, it's way more permissive than Java. Data races are similarly defined: we recall that a data race occurs when two conflicting memory operations that do not operate on "special" variables happens.

The equivalent of a volatile variable is a `std::atomic<T>` to designate a special variable of type `T`. The equivalent of synchronized blocks is defined as `std::mutex(...)`.

7.4.3 OpenMP Memory Model

Since OMP is yet another threading library its memory model differs from C++ 11. There is no concept of a synchronization variable like `volatile` or `std::atomic<T>`. Locks must be accompanied by a flush (ex. to come).

Officially, OMP implements weak consistency except for flush operations.

When we writing code, we need to consider data races only on shared variables.

```
1 #pragma omp parallel
2 private (x) shared (y)
3 {
4     x = ...
5     *y++; //data race
6 }
```

A **flush** is a memory fence.

In order to communicate between threads, they do the following: first thread writes the shared variable, then flushes the variable and the second thread flushes and then reads.

```
1 int y;
2 #pragma omp parallel shared(y)
3 {
4     if( thr_id == 0 ) {
5         y = ...;
6         #pragma omp flush(y)
7     }
8     #pragma omp flush(y)
9     ... = y;
10 }
```

8 Synchronization

Hardware synchronization is the ability if the CPU to provide atomic instructions. For example, in this code:

```
1 // Thread 0, X = 5
2 ld r1, X
3 add r1, 1
4 st X, r1
```

```
1 // Thread 1, X = 5
2 ld r1, X
3 add r1, 1
4 st X, r1
```

Without synchronization it would give us 6 but we expect 7.

Objectives of synchronization:

- There should be low overhead and we have to take care of scalability because synchronization can limit scalability.
- Our synchronization should ensure correctness, synchronization failures are very difficult to debug.
- Hardware and software must agree on synchronizations methods.

Synchronization methods:

- Mutual exclusion: locks.
- Point-to-point: flags, barriers.
- Software methods (not in this course).
- Coarse-grain concurrency control: transactional memory (atomic sections).

Phases of synchronization:

- Acquire method: how threads attempts to gain access to the resource.
- Release method: how to enable others threads to access to the resource.
- Waiting algorithm: how threads for access to the ressource.

8.1 Locks

Before in the course we talked about what to lock, when, ... Now we'll see how locks work.

Locks need to

- Have low latency,
- Add low traffic,
- Be scalable,
- Have a low storage cost.

8.1.1 TS

To implement a lock we need to define a memory location where threads communicate w.r.t. who can execute and who must wait (c.f. *phases of synchronization*).

If we try to creat a lock by just define a variable `lock` that is equal to 0 when the thread can access and 1 otherwise, like this:

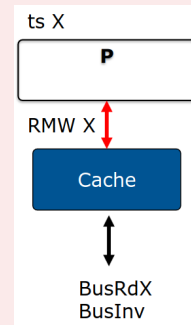
```
1 Lock:
2     ld r1, mem[addr] // load word into r1
3     cmp r1, #0       // if 0, store 1
4     bnz Lock         // else, try again
5     st mem[addr], #1
6 Unlock:
7     st mem[addr], #0 // store 0 to address
```

It won't work because two cores can load 0, thinking the lock is free and then we lose mutual exclusion.

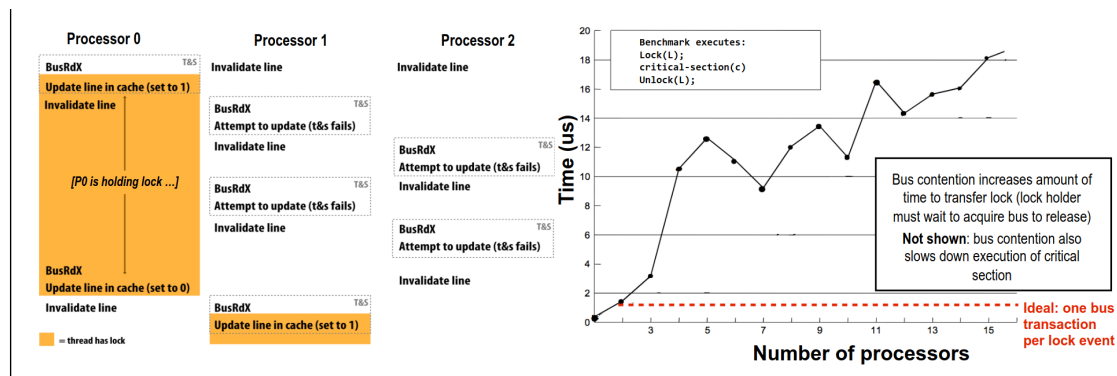
Then we need to have an instruction that ensure atomicity in hardware. We create a new instruction, "test-and-set" (TS): `ts reg, mem[addr]`. A TS atomically load memory location into the register and set the contents of location to 1, so if you are the first to enter, the value will be 1 for others. We can now easily implements a lock:

```
1 Lock:
2     ts r1, mem[addr] // load word into r1
3     bnz Lock         // if 0, lock obtained
4                     // fall-through, critical
4                     // section
5 Unlock:
6     st mem[addr], #0 // store 0 to address
```

We need a new processor-to-cache transaction in cache coherence. When processor execute a `ts` instructions, we send a Read-Modify-Write (RMW) to the cache. Bus transactions between caches will be the same as a Write (W). It means that every TS instruction, other cache lines are invalidate.



We can see now that the problem with TS lock is that the spinning ensure an excessive coherence traffic ! And more processor there is, more traffic increase !



8.1.2 TTS

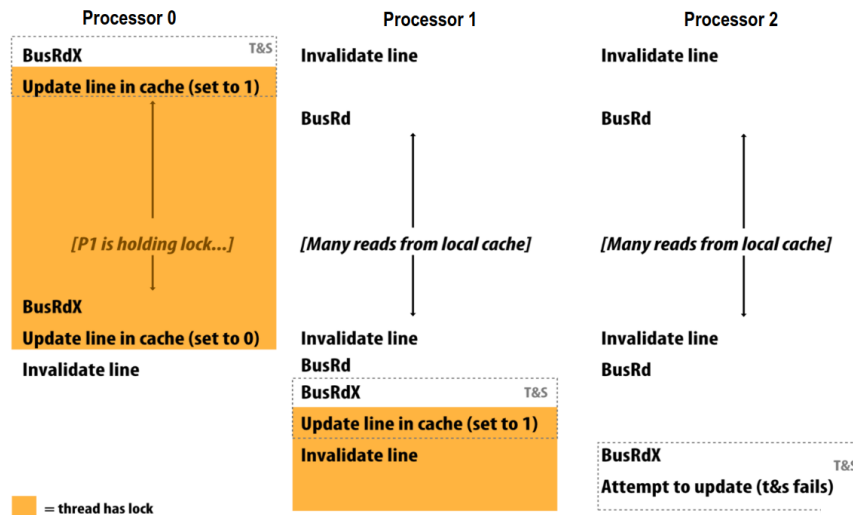
Hopefully there is way to reduce coherence traffic. One way is to test locally.

The idea behind “test-and-test-and-set” (TTS) is to just read classically the lock, if it’s 1 (locked), then we just wait. Then, only if it’s 0, we read one more time the lock with the TS instruction. So when the processor is waiting, it just does `BusRd` instead of `BusRdX` and doesn’t invalidate other caches.

```

1 Test:
2   ld r1, mem[addr] // load lock value
3   cmp r1, #0
4   bnz Test          // compare to 0
5 Lock:
6   ts r1, mem[addr] // load word into R1
7   bnz Test          // if 0, lock obtained
8 Unlock:
9   st mem[addr], #0 // store 0 to
  
```

With TTS we generate much less traffic.



TTS have slightly higher latency than TS, but is more scalable and storage cost stay unchanged.

8.1.3 TS with exponential back-off

Another way to reduce coherence traffic is the TS lock with exponential back-off. The idea is to wait after failing to acquire the lock. To get a balance between lock latency and contention, we increase the time exponentially every fail (1 ms, 2 ms, 4 ms, ...).

TS with exponential back-off have the same uncontended latency as TS but potentially higher latency under contention. It generates less traffic than TS because it's not continually attempting to acquire lock. It also improve scalability but it can cause sever unfairness: newe requester back off for shorter intervals.

One of the issue with TS and TTS is that they require an instruction that is both a read and a write and this is a challenge for pipelined processors. It must also lock the bus until the RMW completes, so it complicates cache controller logic and two memory accesses per sintruction is more difficult to track in CPU pipeline.

8.1.4 Load-locked and store-conditional

Load-locked (LL) and store-conditional (SC) together provides atomi Read-Modify-Write. The idea is that LL read a value a "put it under surveillance", then we can continue to execute some code. After, we can store with SC, that write only if the value hasn't been modified in the memory (by another processor). If it has, we restart from the LL.

We can use LL/Sc to build many types of locks, for example TS:

```

1 Test_and_Set:
2   ldl_l t0, 0(t1)           // load locked, t0 = lock
3   bn t0, L1                // if not free, loop
4   lda t0, 1(0)             // t0 = 1
5   stl_c t0, 0(t1)          // conditional store,
6                           // lock = 1
7   beq t0, Test_and_Set     // if failed, loop
8 Release:
9   stl 0, 0(t1)

```

Blocking synchronization:

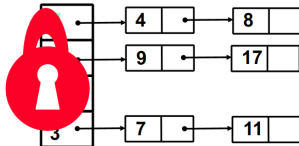
The idea of blocking synchronization is that if the progress cannot be made because a resource cannot be acquired, then we free up the core for another thread. We suspend the running thread, the OS schedules another thread to run and the other thread can run and make progress.

Busy waiting (spinning) vs blocking synchronization:

- Busy-waiting is preferable if scheduling overhead is larger than expected wait time, the scheduling overhead is in order of microseconds where it's in nanoseconds for a few spinning operations.
- Blocking is preferred if waiting time is significant and there are other tasks that need processor's resources.

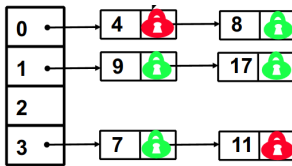
8.1.5 Coarse/fine-grained locking

Coarse grained locks are easy to code, but have bad performance.

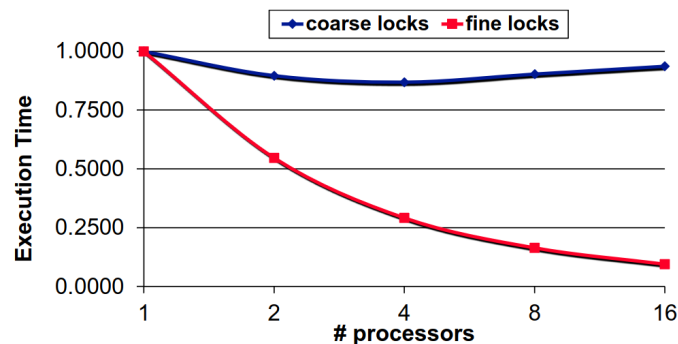


```
1 void editHash(HashTbl tbl, int
  key) {
2     synchronized(tbl) {
3         // read objects
4         HashObj obj = tbl.get(key)
5         ;
6         // update
7         obj.update();
8     }
```

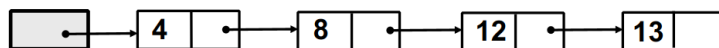
Fine grained locks are harder to code but have reasonable performance.



```
1 void editHash(HashTbl tbl, int
  key) {
2     // read objects
3     HashObj obj = tbl.get(key)
4     ;
5     synchronized(obj) {
6         // update
7         obj.update();
8     }
```



Localized locking: Localizing locking can sometime not be as easy at it seems, on this exemple, which pointers do we edit on `delete(12)`?



The right answer is that we have to lock 8 and 12 because $8 \rightarrow \text{next}$ will now point on 13. To ensure that we get both locks, we grab locks as we traverse the list and we don't release `cur` until we have `cur->next`. Otherwise, another thread could delete it, making our behavior undefined. This is called hand over hand locking.

Locks have other problems!

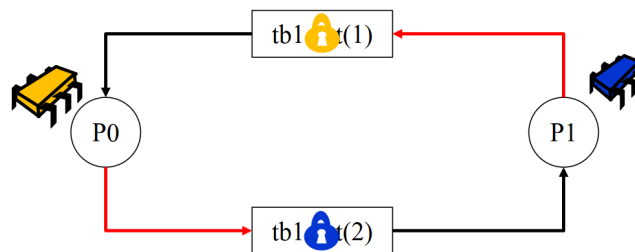
- **Priority inversion:** It's when a low-priority process is preempted while holding a lock needed by a high-priority process.
- **Convoying:** It's when a thread is holding a lock and gets swap out by the OS. Then all other threads that are waiting for the lock need to wait.
- **Lock composability:** It's when a thread contains several locks, it can lead to deadlock or livelock if it's implemented incorrectly.

Here is an example of lock composability which is the major problem in most modern systems. If we have a code that lock successively two objects like this:

```
1 // P0
2 objB = tbl.get(2);
3 synchronized(objA){
4     synchronized(objB){
```

```
1 // P1
2 objB = tbl.get(1);
3 synchronized(objA){
4     synchronized(objB){
```

Then,



9 Concurrency control

9.1 Deadlocks

Many things can block the process, let's focus on deadlocks and livelocks. We already seen what a deadlock is on the *diagram just above*. We can spot deadlocks by looking at what are the necessary and sufficient conditions for deadlocks.

If any of these four properties are missing, deadlock will not happen:

- Mutual exclusion: the system has resources protected by locks,
We can't prevent this property from being true in modern systems, it's essential to prevent data corruption and data races.
- Hold and wait: hold a resource and wait for another,

We saw this example in the *diagram above*.

```
1 objA = tbl.get(1);
2 objB = tbl.get(2);
3 synchronized(objA){
4     synchronized(objB){
5         ...
```

A possible solution is to say that a thread must acquire all resources before it runs. In our example it would be like going from fine-grained locking to coarse-grained locking.

```

1 // Lock the entire hash
  table
2 synchronized(tbl) {
3     objA = tbl.get(1);
4     objB = tbl.get(2);
5     ...
6 }

```

- No pre-emption: there is no way for B to “seize the lock” from A,

This leads to problems such as priority inversion as discussed earlier. A possible solution is to allocate priority to threads so higher priority thread can pre-empt lower priority thread and steal resources. But this approach is generally unsafe, several problems can arise like data corruption or performance overhead.

- Cycling waiting: A waits for B, B waits for A,

This is the easiest and most practical to ensure this property never holds. A solution is to only allow acquiring locks in a fixed order, it means that if two objects A and B needs resources A and B, we say that both will first get (or try to get) A, and only then B. In our example, both processors would have `objA = tbl.get(1);`.

Challenges and applications:

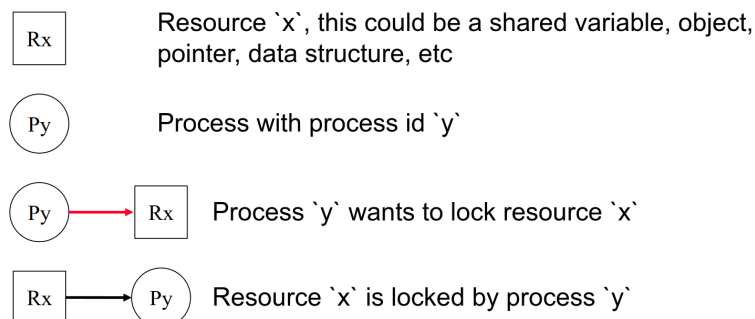
Processes often don't know ahead of time which resources to lock, the programmer must ensure code follows ordering. Ordering locks often applied to list-like data structures, when the elements to lock are bounded, for example: multiple threads accessing a single array, sort the elements of the array in some order and allow threads to lock the elements one by one in the order. This is a very popular approach that we found inside the linux kernel.

9.1.1 Detection

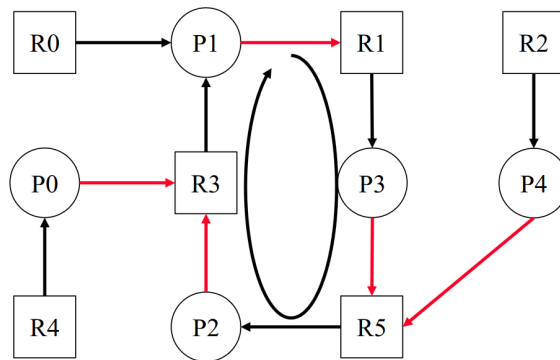
Sometimes we can't prevent deadlocks, then, the second best approach is to detect it and kill processes involved in the deadlock. This approach requires two things: an algorithm to detect deadlocks and an algorithm to recover from a deadlock.

A **resource allocation graph (RAG)** is a graph where: nodes are either resources or threads, a resource node can have multiple units and there is directed edges between threads and resource. If RAG has no directed cycle, the current state is deadlock-free. If RAG has a directed cycle: if there is only one unit per resource, there is a deadlock, for multiple units per resource, there is a possibility of deadlock.

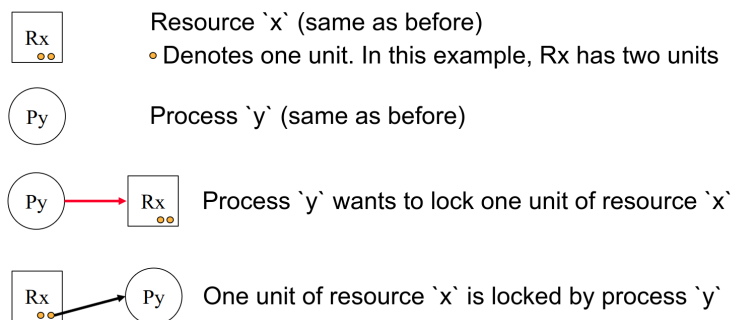
Let's see the notation of the one unit per resource case.



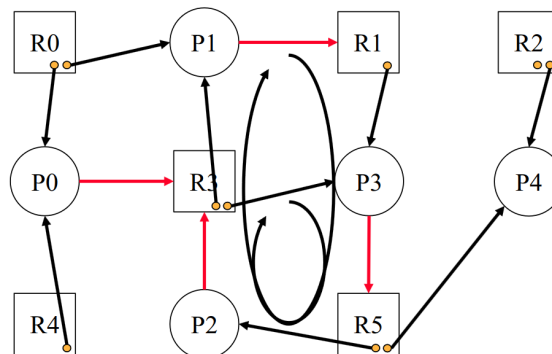
On this graph, we can see that there is a directed cycle, then the system is deadlocked!



Let's see the notation of the multiple unit per resource case.



In this graph there is two directed cycle, but there may or may not be a deadlock. Let's check.



Let's go step by step:

1. P4 can execute and free up resource, we can remove it from the graph,
2. P3 can now use R5, which has free unit,
3. P3 will execute next and can be removed from the graph,
4. P1 can now use R1 and P2 can use R3,
5. Both P1 and P2 can execute and be removed from the graph,
6. Finally P0 execute.

So there is no deadlocks in this system!

Now that we know how to detect a deadlock, what to do next? We can either terminate the process or pre-empt it.

- Process termination is easy to do (simply kill all deadlocked processes) but may run into

data corruption issues.

- Process pre-emption is harder to do (must choose a fair victim and how much to rollback) but a more safer approach.

Livelocks have the same four necessary and sufficient conditions for deadlocks apply.

9.2 Transactional memory

Instead of writing difficult locking code, we can simply tell the runtime to make a section atomic. This is thanks to the hardware transactional memory (HTM). We only need to declare a variable so its updates will be atomic, language runtime figures it all out.

A **memory transaction** is an atomic and isolated sequence of memory accesses, it's supported in hardware by processor and cache hierarchy.

Transactional memory (TM) have several properties:

- Atomicity: upon transaction commit, all writes take effect at once and on transaction abort, no writes take effect.
- Isolation: no other processor can observe writes before commit.
- Serializability: transactions seem to commit in a single serial order but the exact order is not guaranteed.

Contrary to locks, transactions compose gracefully.

We need some new instructions and hardware changes to implement TM.

Let's introduce three Intel Haswell instructions: `xbegin` → start transaction, takes pointer to fallback address in case of aborts, `xend` → ends transaction, `xabort` software initiated transaction abort.

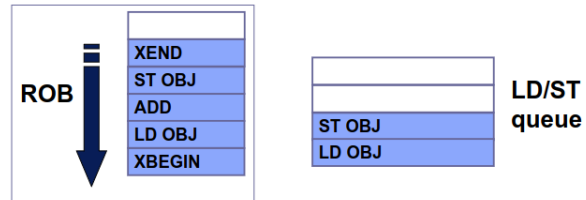
The CPU assume that no conflicts occur, it records every read and writes between `xbegin` and `xend` and every state is speculative (can be seen only by the core). Then the CPU checks for conflicts in the data structure (if another core is using same resources), for example, it sees a BusRdX or a BusInv matching a load (W/R) or a store (W/W) in LSQ. Check if there is another corner case (cache eviction, interruption, ...). Finally, on conflict, execute recovery code.

Here is an code exemple

```
1 xbegin recover
2 ld r1, mem[obj]
3 addi r1, #10
4 st mem[obj], r1
5 xend // commit trans.
6 recover:
7 br rec_handler
```

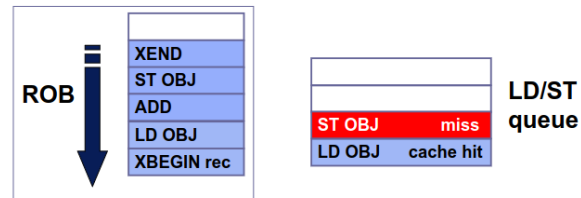
We can observe two cases, the first one, successful:

LD OBJ and ST OBJ hit in the cache, all state remains in the pipeline and we wait until we see XEND to start committing.



Let's see now a case with conflict:

Here we assume that OBJ is in S state. Then the store operations induce sending a BusInv to get to M state (can be called a store miss, we can't write immediately). We then flush all instructions and execute the jump to label "recover" given by the user.



So we have to check for conflict at every operation by using coherence actions (BusRd, BusRdX, BusInv, etc...).

9.3 Point to point synchronization

Let's recall flag-based synchronization:

Thread 0:

```
1 // val = done = 0;
2 val = expensive_func();
3 done = 1;
```

Thread 1:

```
1 // done = 0;
2 while(!done){ }
3 ... = val;
```

It assumes sequential consistency!

Barriers introduce planned stalls in threads, like wait for all threads to read input before start. They use flags.

Here is the idea behind the implementation of the barrier:

1. Clear the flag, tell all incoming threads to wait,
2. Every arriving thread increments counter,
3. Last thread sees counter == num_threads,
4. Set the flag, all threads leave,

```
1 struct Barrier_t {
2     Lock aLock;
3     int counter;
4     int flag;
5 };
```

A basic implementation of this algorithm wouldn't work because some thread can arrive to a new barrier while others haven't left the one before, so there would be a deadlock. That's why we need to keep a private copy of the flag, the first thread to arrive writes its private copy (`int local_sense = 0; //private`). It looks like this:

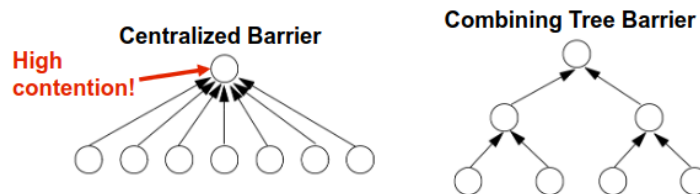
```

1 void senseBarrier(Barrier_t* b, int num_threads) {
2     local_sense = !(local_sense); // (Step 1)
3     lock(b->aLock);
4     int num_waiting = ++(b->counter); // (Step 2)
5     if( num_waiting == num_threads ) { // (Step 3)
6         unlock(b->aLock);
7         b->counter = 0;
8         b->flag = local_sense; // (Step 4)
9     } else {
10         unlock(b->aLock);
11         while( b->flag != local_sense ); // spin
12     }
13 }

```

Traffic on bus: $O(P)$, **Latency:** $O(P)$.

But we can achieve a $\log(P)$ latency with **combining trees**! The strategy is to acquire when the processor arrives at barrier, performs `atomicIncr(parent.counter)`, and the process recurses to the root. The idea is that when every children of a parent are arrived, the parent (who is also a children) can increment its own parent, until we reach the root.



Examples of locks in OpenMP:

- `omp_init_lock(omp_lock_t *)` : initializes the lock variable passed, lock is not held by the initializing thread,
- `omp_destroy_lock(omp_lock_t *)` : disassociates the given lock variable from any locks,
`omp_set_lock(omp_lock_t *)` : waits until the lock is available, then acquires the lock,
- `omp_unset_lock(omp_lock_t *)` : releases the lock,
- `omp_test_lock(omp_lock_t *)` : attempts to set a lock, but does not block if the lock is unavailable.

```

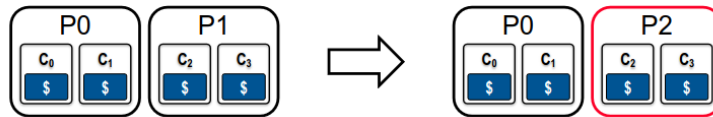
1 omp_lock_t lock;
2 omp_init_lock(&lock);
3
4 #pragma omp parallel num_threads(4)
5 {
6     int tid = omp_get_thread_num( );
7     int i;
8     for (i = 0; i < 3; ++i){
9         omp_set_lock(&lock);
10        printf("T %d: begin locked region\n", tid);
11        printf ("T %d: end locked region\n", tid);
12        omp_unset_lock(&lock);
13    }
14 }
15
16 omp_destroy_lock(&lock);

```

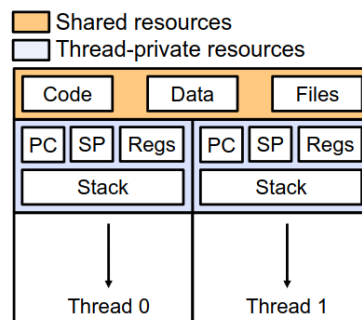
10 Context switching

A **process** is running instance of a program and are independent of each other. Each process has its own virtual address space and they don't share any system resources.

Hundreds of processes can exist on a CPU but only few of them run in parallel at any given time. The OSS schedules and switches processes as needed, it gives the illusion of multi-tasking and allows for concurrency (another process runs when one is blocked). For now, P in picture means process not processor.



A single process usually runs multiple **threads** at the same time. Threads are independent and can execute various functionality, each has its own program counter, stack and variable values. Unlike processes, threads also share resources: the heap, the code and global data of the process are shared. All threads operate in the same virtual address space.



We already saw OpenMP, which is simply an easy-to-use abstraction over Pthreads.

POSIX threads (Pthreads) are general-purpose threads. They are created, scheduled and managed by the kernel, these are also referred to as “kernel” threads. They offer fine-grained control over functionality, synchronization and scheduling.

Many languages offer high-level thread libraries using Pthreads, `thread` library in C++ for example, that can be used to build complex multi-threaded programs. Threads can also switch to avoid wasting cycles.

For example, in a web server, we can create a new thread everytime that we need to handle a new request, so thread 0 can continue to loop and listening for incoming requests.

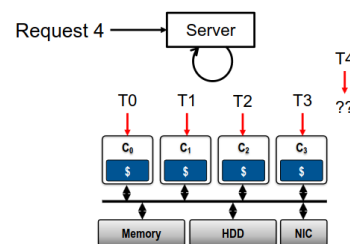
Here is the code example:

```

1 struct Request {
2     ...
3 };
4 void handle_request(Request req) {
5     ...
6 }
7 int main() {
8     while (true) {
9         auto req = get_next_request();
10        if (req) {
11            std::thread t(handle_request, *req);
12            t.detach();
13        } else {
14            std::this_thread::sleep_for(std::chrono::
15                milliseconds(50));
16        }
17    }
18    return 0;
19 }

```

But now imagine we are running the web server on a CPU with four cores, at any given time, only three requests can be handled in parallel.



If there is a high rate of incoming requests, the web server will spawn new threads to handle the new requests. These new threads will be locked behind the threads handling the old requests. Then, there is two options: wait for the threads executing the old requests to finish first or context switch threads!

Let's see that requests can waste cycles. In this code,

```

1 struct Request {
2     int id, type;
3     std::string payload;
4 };
5
6 int hash_arr[N];
7 void handle_request(Request req) {
8     if (req.type == 0) {
9         std::ofstream outfile("output.txt", std::ios
10             ::app);
11         outfile << "Request " << req.id << ": " <<
12             req.payload << "\n";
13         outfile.close();
14     } else if (req.type == 1) {
15         hash_arr[req.id] = calc_hash(req.payload);
16     }
17 }

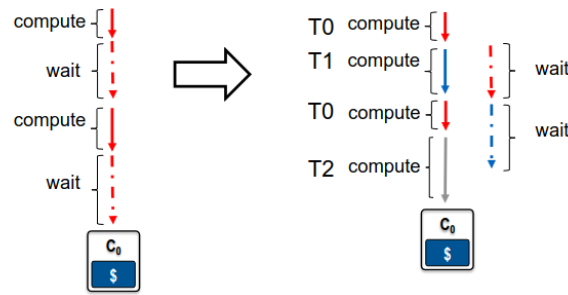
```

We have two type of request: type 0 write the string into a file, type 1 compute the SHA hash of the string and store it. Writing a single line to a file can take between hundred μs to ms . So a thread handling a type 0 request is simply IDLE during I/O $\rightarrow$ more than hundred μs are wasted per line.

By this time we could serviced around ten type 1 request. There is also threads executing type 1 requests that can get blocked behind threads executing type 0 requests that are wasting clock cycles.

We already saw that parallelism is the ability to execute threads at the same time, to speed up a single task by splitting it amongst multiple threads. It exploits multiple cores. But what is really concurrency?

Concurrency is the ability to hide latency among blocked threads, to increase utilization of a core and throughput.



For our code with context switching the kernel would switch the thread with a “ready” thread. Once the I/O operation is done, the kernel switches back the waiting thread. The kernel co-ordinates the switch:

1. It saves the context of the current thread,
2. Calls the scheduler algorithm to pick the next thread,
3. Restores the context of the next thread.

The context saved means that we save the “state” of a thread that must be saved before a switch.

You need to remember the PC (program counter) of the last executed instruction before the switch, here `lw r6, 0(r3)`, and also the value of all registers before the switch.

```

1 lw r1, 0(r2)
2 lw r6, 0(r3)
3
4 /* Switch to a different
   thread */
5 ...
6 /* Resume normal execution */
7
8 mul r1, r1, r4
9 add r6, r1, r6
10 sw r6, 0(r3)

```

There is no need to save shared resources, only the thread-private ones.

But context switching involves overhead. The explicit one: a syscall means letting the pipeline empty, use the trap table to jump to the syscall code, switch OS. We need also to dump the current thread’s context to memory, load the new thread’s context and return from syscall, switch back to user. But there is also the implicit overhead: all microarchitectural state that is shared (branch tables, TLB, cache hierarchy) is affected while a thread is running. Considering all this overhead, context switching is only worth if the time to switch, run and switch back is less than IDLE time, knowing that a single switch between threads in Linux takes 1-5 μ s.

10.1 User threads

Only the kernel has permissions to schedule threads to run on cores. But using the kernel for switching is overkill. So an idea is to separate the two requirements:

1. Scheduling threads to run on cores - the kernel is needed.
2. Switching between threads on the same core - we can actually do this on user threads.

The creation of a thread is done by the function `pthread_create()` that uses a syscall to ask the kernel to create the thread. To create a thread we need to:

- Allocating a data structure to store the context (PC, SP, regs).
- Allocating a stack.
- Initializing the context.

So now what if the user created threads instead of the kernel. A user thread library would handle tasks typically done by the kernel: create and manage threads, schedule which thread runs and when, save and restore context during switches.

A thread created entirely in user space is called a **user thread**. Thread scheduling happens then at the user level.

At any given time, only one user thread can run on a core: the user thread either runs to completion or gets blocked. If the user thread gets blocked or completes execution, it needs to switch to another user thread. Previously, it was the kernel who detected blocked or completed threads, now it's not possible because the kernel is unaware of these user threads.

So now the user thread needs to **yield** to give back control. It means that it should give up control of the core voluntarily. The user thread library has a scheduler which picks the next user thread to run.

Here is a pseudocode example of a context switch with user threads. We first create a new thread by allocating and initializing the context and add the thread to a global queue. Then the function `yield` saves the context of the current thread, appends it to the queue, gets the next thread from the queue, restores the context of the next thread and resumes execution of the next thread.

```
1 function create_thread(func):
2     thread.context, thread.stack = init()
3     thread.function = func
4     queue.append(thread)
5
6 function yield():
7     save_context(current)
8     queue.append(current)
9     next = queue.pop()
10    restore_context(next)
11    resume next.function()
12
13 function func1():
14     ...
15     yield()
16
17 function func2():
18     ...
19     yield()
```

Then, we done context switching without the kernel!

So switching is 10-100 times faster than switching between kernel threads because there is no overhead from syscalls and kernel scheduler. We can then make more switches and have a better concurrency.

10.2 Coroutines

The yielding needs to happen at the same time as the operation that will block our thread. We have to switch before that the part that blocks in the function is executed.

The idea of **coroutines** is that the programmer knows when something is going to block the thread so he has functions that can cooperate by pausing and resuming its own function. Cooperative functions or routines are called **couroutines**: they are functions that can be paused and resumed. They are implementable through callbacks and language support.

Let's switch now to retake our *web server* code but now in Python:

```
1 def save_file(req):
2     with open('output.txt', 'a') as file:
3         file.write( f Request {req.id}: {req.
4                     payload} )
5     return
6 def sha(req):
7     hash_arr[req.id] = calc_hash(req.payload)
8     return
```

The `save_file` function execute *type 0* requests and the `sha` function execute *type 1*. The web server is running on a single core, on the arrival of a request, the corresponding function gets called.

Implement coroutines requires changing how functions are executed. A normal function call work like this

Function call $\implies$ Function $\implies$ Result

To implement a function defined as a coroutine we have to use `async`:

`def save_file` becomes $\implies$ `async def save_file`

Then the programmer pauses the function using `await`:

```
1 async def save_file(req):
2     with open('output.txt', 'a') as file:
3         file.write( f Request {req.id}: {req.payload} )
```

becomes

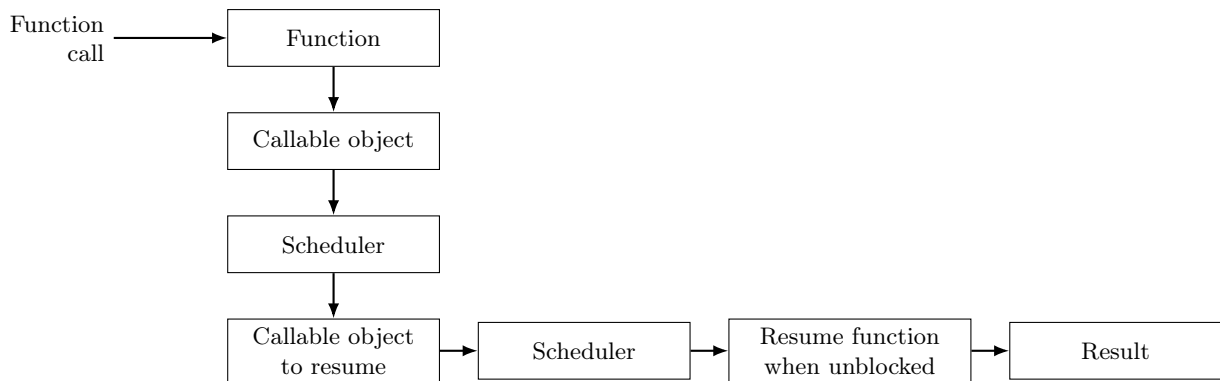
```
1 async def save_file(req):
2     with open('output.txt', 'a') as file:
3         await file.write( f Request {req.id}: {req.
4                           payload} )
```

The `await` keyword tells Python to pause executing this function until the write is finished and resume on completion of the write.

Remark

Note that `await` should only be used on other asynchronous functions.

Calling a coroutine function does not immediately execute it, it returns a coroutine object (an “awaitable” in Python) that can be used by the internal Python scheduler that decides when to run and resume execution.



Remark

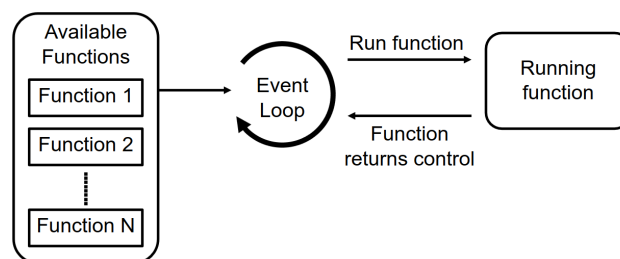
Note that the context is saved in an internal data structure in the user space.

In Python, it is standard to run these functions in a loop on one thread: the internal Python scheduler chooses which function to run, when one function gives up control, another ready function will take control. Here an example of a simple code in Python:

```

1 import asyncio
2 import random
3
4 async def say_hello(index):
5     await asyncio.sleep(random.uniform(0, 2))
6     print( f Hello world, number {index} )
7
8 loop = asyncio.get_event_loop()
9 loop.run_until_complete(
10     asyncio.gather(say_hello(0), say_hello(1),))
  
```

The loop that manages all coroutines is called the **event loop**, it waits for events (like I/O, timer, messages) and pauses and resumes functions on the completion of events.



Let's now adapt our *server*. We have to first modify our function `save_file` and `sha`: we have to turn the asynchronous, we have to make the I/O write function inside asynchronous too and finally we need to wait on the file writing operation. The our functions look now like this:

```

1 async def save_file(req):
2     async with aiofiles.open('output.txt', 'a') as f:
3         await f.write( f Request {req.id}: {req.payload} )
4
5 async def sha(req):
6     hash_arr[req.id] = calc_hash(req.payload)
  
```

Now, we need to have an event loop to handle incoming requests:

```

1 async def handle_request(req):
2     if req.type == 0:
3         await save_file(req)
4     elif req.type == 1:
5         await sha(req)
6
7 async def event_loop():
8     while True:
9         req = await get_next_request()
10        if req:
11            asyncio.create_task(handle_request(req))
12        else:
13            await asyncio.sleep(0.05)

```

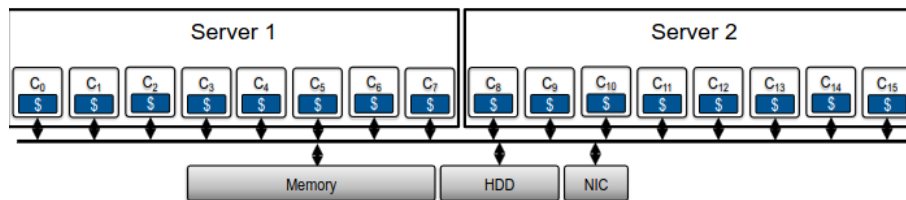
11 Microservices

We did a server and ran it with only one executable file, one application deployed even if it's running on four cores. It is called a monolith.

A **monolith** is a single application that contains all the functionality. All functions are part of the same program and run in the same address spaces.

Real-world monoliths are complex and contain several functions, it suffers from synchronization overheads among threads, it doesn't scale well. Overhead limits monoliths from fully utilizing all the cores and there is a synchronization limit on performance beyond a few cores. In resume, there is a lot of part that are dependent and should be synchronized.

A solution to use cores effectively is to run multiple monolith copies. Each instance behaves as an independent server, threads among different instances shouldn't synchronize with each other. Also, each instance runs on a smaller number of cores and scales well.



But monoliths use other resources too:

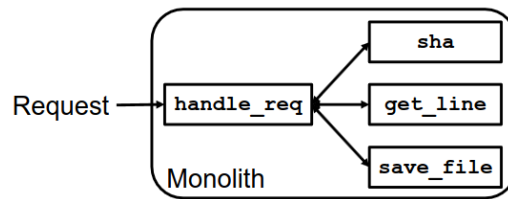
- Memory capacity: each monolith can take a big chunk of server's memory.
If each instance of a server can consume a large amount of memory (e.g. 60% of the system's total memory capacity), then you can't run many instances together on the same server.
- Memory bandwidth: Analytic workloads scale well but use a lot of memory bandwidth.
- NIC bandwidth: performance may be limited based on availability of NICs.
- Accelerators: GPUs in a server may require CPU cores for coordination.

A monolith does not provide strong isolation between functions: for example, a third-party cryptographic function like `sha` and an internal function like `save_file` run inside the same process and cannot be executed separately, since they share the same runtime and resources, which tightly couples them at runtime.

Now that we saw every disadvantages of monolith, how can we improve this?

In our web server, we can maybe group our functions based on the type of task they perform. We clearly have to distinct types of tasks:

1. Save and serve files (`save_file` and `get_line`)
2. Perform hash computations (`sha`)



These two tasks are independent, they can be separated logically and physically !

Groups of functions can be treated as separate “services” based on functionality and needs. Services have separate codebases, address spaces, etc. . . This is called a **microservice architecture**: a set of small (micro), independent and self-contained services.

A service executes a single task type in isolation, each service has its own address to receive requests. Each service has also its own copy of `handle_request`, processes incoming requests and chooses which function to execute.

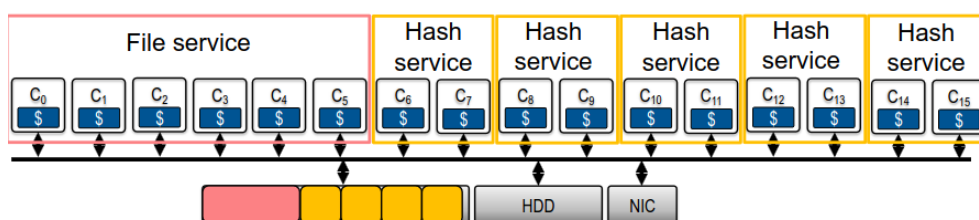
Monolith	Microservices
One big program containing all functions	Functions are grouped into small programs (services)
All functions run in the same address space	Each service runs in its own address space
Functions call other functions and use shared memory to communicate	Functions in various services can also “call” each other and communicate using messages (in detail later)

Services have various system requirements: e.g. `file.py` (to handle file) will use more memory but `hash.py` (to crypt) will use more compute.

Services are not used uniformly: some services are more popular than others, they will handle more requests.

With microservice each service can be scaled independently of the others: popular services can be scaled without scaling the other services and resources can be better allocated depending on each service’s needs.

Before, we saw that if our server use eight cores and 60% of the system’s total memory then we can only run one server. Now we can completely utilize all cores and memory:



Other’s advantages are:

- Services can be written independently of each other and we can choose the best program-

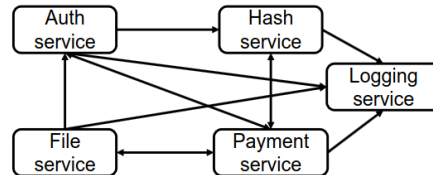
ming language according to a service's needs.

- We can have cheaper plans: cloud providers offer servers with different price in function of resources that they have (e.g. memory).
- There is more fault tolerance: one service failing doesn't lead to the entire workload failing.
- It allows reusability: services are isolated but can be accessed from any other service.

But there is also disadvantages! Services have to communicate between them by messages, it becomes critical as the number of services increases.

Remark

Connectivity graph are important for scalability:



The communication protocol of microservices must be:

- fast,
- fault-tolerant,
- able to locate each service on the network,
- able to pass input on the service and receive its output.

11.1 RPC

Microservices communicate as if via function calls.

There is local calls that are calls by a function to another function within the same service. This is our conventional function call. We call this **local procedure call (LPC)**.

There is remote calls that are calls by a function to another function in a different service. This is new and used for microservices. We call this **remote procedure call (RPC)**.

To first define our function, we use an interface definition language (IDL): this language will not run any code, it's a common format to describe functions across languages.

We first have just to define the interface of our function, not an implementation:

```
1 service hash{
2     rpc sha(Request) returns (StringValue) {}
3 }
```

The RPC library has implementation for various languages like C, Python, etc. . . It will take in the interface definition code as input and will generate code in the language of all our services from one interface definition code.

The code generated for the service that implements and serves a function is called the server stub. The server stub will just be an abstract function, the developer have to implements it.

```

1 // generated by rpc library in C for the hash
  service
2 class HashService::service{
3     // Abstract function, later implemented by
      developer
4     virtual char * sha(Request req)= 0;
5 }

```

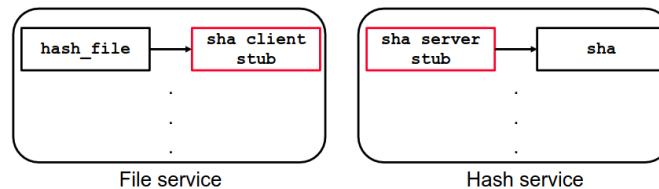
The code generated for the services that use a function is called the client stub. This is a concrete code that RPC will implement sending the request to the service that serves it.

```

1 # generated by rpc library in Python for the file
  service
2 class HashService:
3     def sha(req: Request)-> String:
4         # RPC sending the request to hash service
5         resp = sendReq(req)
6         ...

```

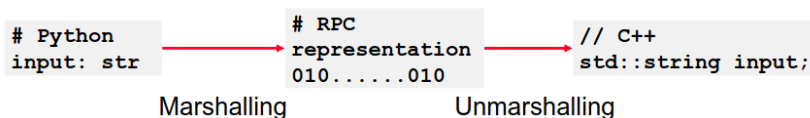
In our server case, the `hash_file` function is using `sha` function. So it will look like this:



We can't directly transfer data between stubs, because languages stores types in various formats in memory. So we have to do marshalling/unmarshalling.

Marshalling turn data from a memory representation into a representation that can be sent over the network. RPC unifies marshalling data for all servicees, once received RPC needs to be able to save it to memory again.

Unmarshalling turn data received from the network in an IDL representation into a memory representation of the receiving language.

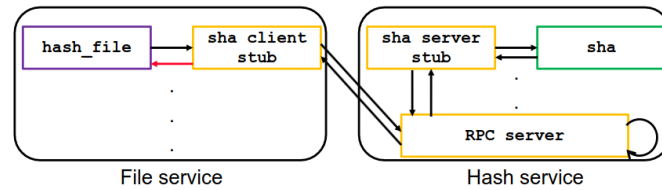


To call a function, for example `hash_file` that calls `sha`, it works like this:

1. `hash_file` calls the client stub's `sha` function with its inputs.
2. Client stub's `sha` function takes the input and marshalls data from memory into a representation that can be sent over the network (RPC provide the send function to send marshalled data).
3. Marshalled data is sent to hash service, RPC libraries include a built-in server that listens for RPC requests coming in from the network.
4. Server stub's `sha` function takes the input and unmarshalls the data into variables that can be used by functions.
5. It calls the functions `sha`.
6. Server stub marshalls the result and sends it back to the caller.

7. Client stub unmarshalls the result and returns it to `hash_file`.

All these steps are generated by RPC.



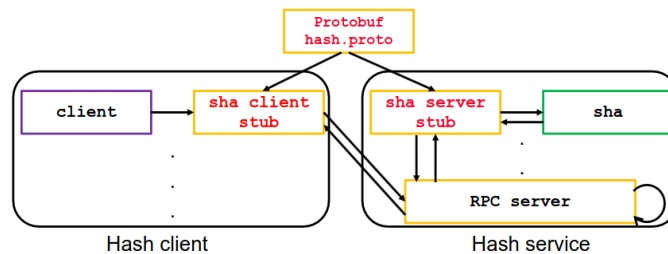
```

1 # generated client stub by rpc library for file service
2 class HashService:
3     sha(self, input):
4         req = marshall(input)
5         resp = send(req)
6         return unmarshall(resp)

```

11.1.1 gRPC

There is multiple RPC, we'll see for example gRPC that uses protocol buffers (protobuf) as IDL.



The code of declaration of function would then look like this:

```

1 // hash.proto
2 syntax = "proto3"; // Define protobuf syntax
3
4 message HashRequest {
5     string input = 1; // One is the ordered attribute
6     // for seialization
7 }
8
9 message HashResult {
10    string result = 1;
11 }
12
13 service Hash {
14     rpc GetHash (HashRequest) returns (HashResult);
15 }

```

IDL's need to be compiled to be used, here protoc is the official protobuf compiler.

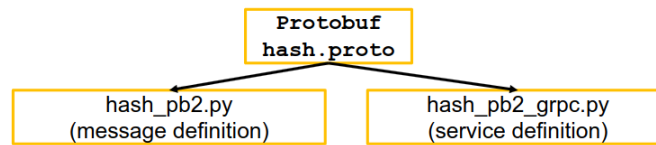
```

1 pip install grpcio-tools // install compiler
2
3 python -m grpc_tools.protoc \ // starting it
4     -I . \ // IDL folder
5     --pyi_out=. \ // output destination
6     --python_out=. \ // output destination
7     --grpc_python_out=. \ // output destination
8     hash.proto //IDL file

```

Each .proto file will generate a `_pb2.py` file that contains the definition of all inputs and outputs and implements marshalling and unmarshalling for inputs and outputs.

It will also generate a `_pb2_grpc.py` file that contains the definition of the services: the abstract class to be implemented in the server and the stub class to be used as substitute by clients.



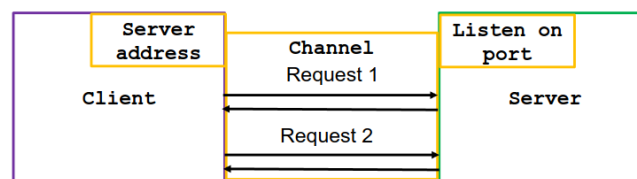
Now that we have define all this, we can create our server using gRPC

Server creation using gRPC:

```

1 # hash_server.py
2 from hash_pb2_grpc import HashServicer //
3   HashServicer is the "server stub"
4 from hash_pb2 import HashRequest, HashResult
5 class HashServiceImpl(HashServicer):
6     def GetHash(self, request:HashRequest, context):
7         hash_result = HashResult()
8         hash_result.result =str(hash(request.input))
9         return hash_result
10 from concurrent import futures
11 request_thread_pool = futures.ThreadPoolExecutor(
12     max_workers=10)
13 import grpc
14 server = grpc.server(request_thread_pool)
15
16 from hash_pb2_grpc import add_HashServicer_to_server
17 service_implementation = HashServiceImpl()
18 add_HashServicer_to_server(service_implementation,
19     server)
20
21 server.add_insecure_port(':::50051')
22 server.start()
23 print("Hash server is running on port 50051 on local
24     host...")
25 server.wait_for_termination()
  
```

Usually we should have done a TCP connection to communicate with the client. But gRPC channels abstract away communication, everything is built-in: streaming, retries, deadlines and cancellations.



Now that we have our server, we have to create a client. We then need a stub as a substitute for server's services: it can be used instead of local procedure calls. We also need an address to where server is listening for requests (channel)

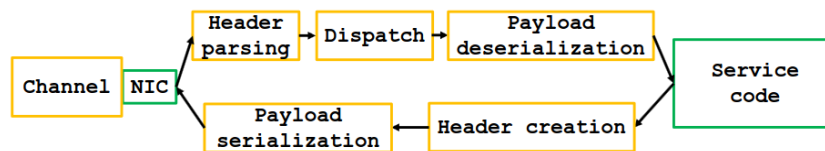
```

1 # hash_client.py
2 import grpc
3 channel = grpc.insecure_channel('localhost:50051')
4
5 from hash_pb2_grpc import HashStub
6 stub = HashStub(channel)
7
8 from hash_pb2 import HashRequest
9 request = HashRequest(input="Hello world!")
10
11 from hash_pb2 import HashResult
12 response: HashResult = stub.GetHash(request)
13 print(f"Hash result for '{request.input}' is: {
    response.result}")

```

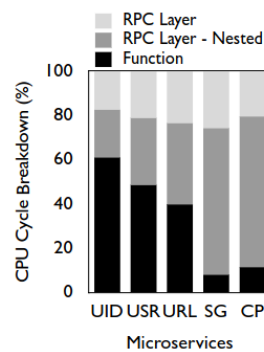
RPC and gRPC are everywhere:

- In the industry, it is used by google cloud platform's services, LLM agents...
- In hardware, it is used to transports message to NIC:



RPC layer takes 40-90% of microservice time.

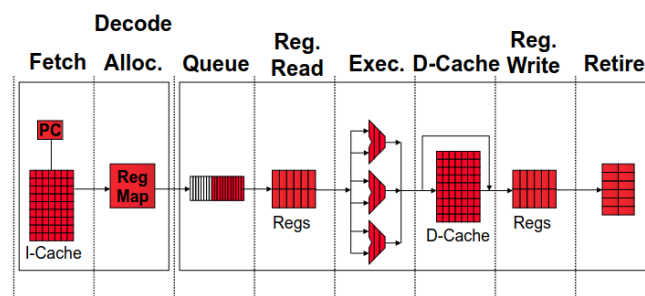
Reduce CPU time on RPC overheads is an active area of research.



12 Multi-threaded processors

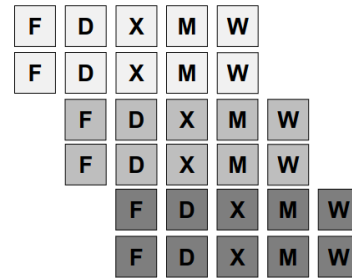
Arithmetic units in CPUs are often not busy because there is long-latency memory accesses, exceptions due to I/O or syscalls, etc... Then, we often switch threads a lot. The thing is, can we keep thread context around in hardware instead of save and restore context everytime?

Let's remind how looks a basic superscalar pipeline:



Pipelining allows multiple instructions to be in-flight:

Assuming 5 stages (F, D, X, M, W)
and a perfect
pipeline $\rightarrow$ scalar IPC = 1.
On superscalar, we can duplicate the
pipeline $\rightarrow$ IPC = 2.



But, in reality, there can be pipeline bubbles that decrease IPC. They can occur for multiple reasons, one is **structural hazards**: not enough resources are available and instructions are queued, waiting for resources to free up.

For example, in a 2-way in-order superscalar processor with one cache port (or one load or store per cycle), if two loads align in same cycle, second one must wait.

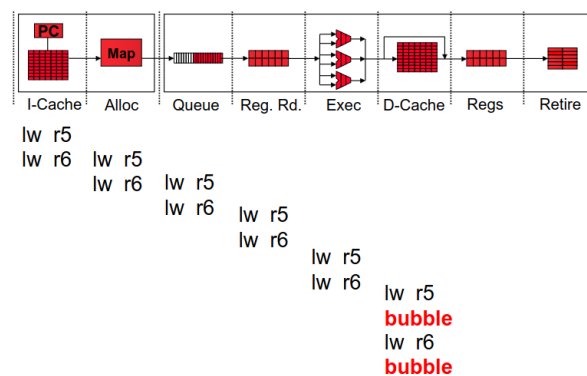
If we take the same setup we can see
this in this code for example:

```
1 for (i = 0; i < N; i++)
2   a[i] += (b[i] + c[i])
```

That would look like this in
assembly:

```
1 loop:
2   lw r5 , 0(r2)
3   lw r6 , 0(r3)
4   lw r7 , 0(r4)
5   add r5 , r5 , r6
6   add r5 , r5 , r7
7   sw r5 , 0(r2)
8   addi r2 , r2 , 4
9   addi r3 , r3 , 4
10  sub r9, r8, r2 ;
11  addi r4 , r4 , 4
12  bne r9, loop
```

Then, because we have only one cache port, we would have bubble on the load operations:



And our pipeline would have bubbles (also caused by the add waitin for **sw** operation):



Finally we obtain an average IPC of 1.6 ($\frac{\# \text{ instrs}}{\# \text{ cycles}} = \frac{11}{7} = 1.6$).

An other reason why pipeline bubbles occurs is the **branch controle flow**: it's when we predict the branch direction to keep fetching instructions.

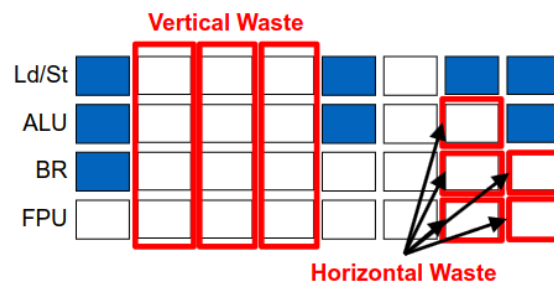
If we are waiting for the real direction before fetching it would introduces bubbles: two cycles minimum with a shallow pipeline because we usually know the target of the branch (e.g. `beq`) on the execute stage. Real pipelines are much deeper (14+) so we prefer to predict the target of the branch to continue to fetch immediately. If the prediction was right we continue, otherwise we flush the pipeline.

Finally data dependences can also obviously create bubbles in the pipeline: they can't execute in the same cycle, etc... But we remember that with OoO cores, we can easily traverses loops and issues loads.

But real programs are not so simple: there is many cycles where nothing can be issued a more realistic IPC would be between 0.5 and 1.

We can imagine then that there is a lot of waste, we classify it in two categories:

- **Vertical waste**: whole cycle is empty, nothing issued, this is common after long latency events.
- **Horizontal waste**: we are unable to use full issue width, this is when the software isn't exposing enough ILP for hardware (the code isn't enough parallelizable).



To solve this, the idea of **hardware multithreading** is to find ready instructions from many threads. But it requires to have thread context (PC, SP, register file, interrupt descriptors) in hardware.

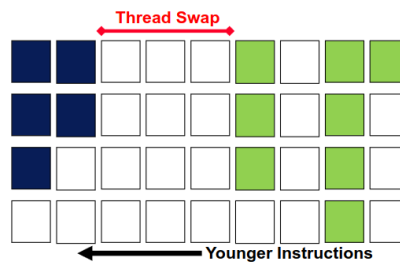
So we said that we have to store the context, so it has a hardware cost: each thread needs architectural state with a register, the PC, the stack and the interrupt tables. It costs SRAM and can impact area especially for small cores.

Threads are also sharing the memory hierarchy now:

- It causes potential contention in the data cache,
- It causes contention in instruction cache,
- It will affect single thread performance (better throughput but worse latency).

12.1 CGMT vs. FGMT

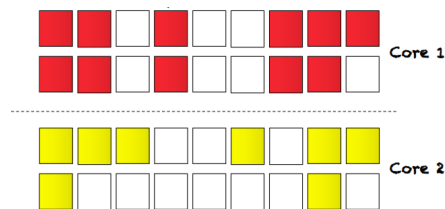
The **coarse grained multithreading implementation (CGMT)** consist in switching to a new thread on a long latency event.



Remark

We can see that the real difference of hw multithreading with user-level/OS context switching is that it takes only few cycles instead of multiple milliseconds.

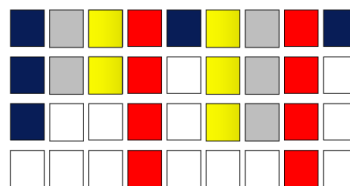
If we made the choice of not having a 4-way pipeline with hardware multithreading but two cores with 2-way pipeline, it would have less horizontal waste but it would suffer from more vertical on long latency.



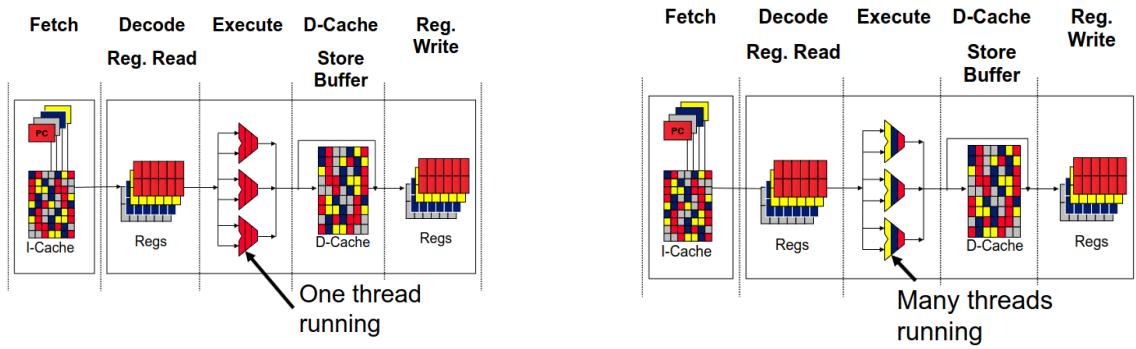
The critical decision on CGMT is when we have to switch threads: the thread switch + the pipe fill thime should be way smaller that the blocking latency.

The advantage of CGMT is that it requires small changes to exisiting hardware but it slow down the performance of a single-thread (shared resources, etc...).

The **fine grained multithreading implementation (FGMT)** consist in cycle between multiple threads periodically, we eliminate the switch time by keeping all thread hot.



If we compare both CGMT and FGMT it looks respectfully like this:



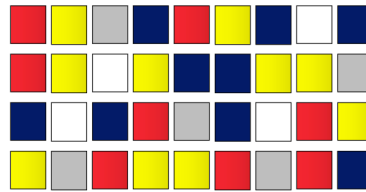
In FGMT design there is no critical decision, to improve throughput we only have to have enough threads to eliminate vertical waste (e.g. for a pipeline depth of 8, using 8 threads isn't enough).

FGMT have a reasonable single thread performance and is efficient in case of many thread. But it's more complicated for hardware to track dependences and many more contexts means we need many more registers.

Notify that both, CGMT and FGMT cannot address horizontal waste, only vertical.

12.2 SMT

The idea of **simultaneous multithreading (SMT)** is to have instructions from multiple threads in the same cycle. This is the first design that can reduce horizontal waste.



Previously, only caches and predictors were shared, with SMT, instructions from different threads are interleaved in the same cycle.

But do we need then to resolve all potential dependences between threads (e.g. T_1 and T_2 are both executing `add r1, r2, r3`)? No we don't because registers are physically mapped then,

T_1 : `add r1, r2, r3` becomes: `add p4, p2, p3`

T_2 : `add r1, r2, r3` becomes: `add p9, p8, p5`

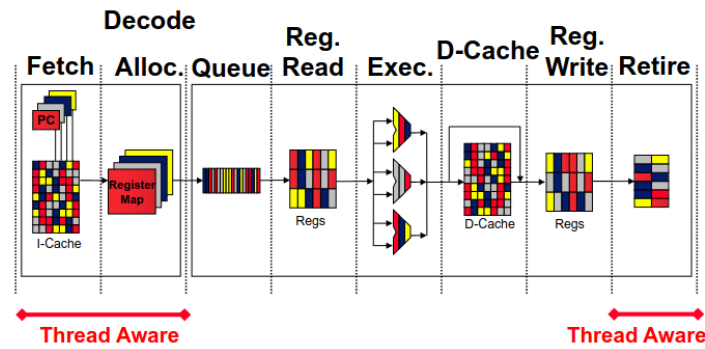
After instruction allocation, the scheduler becomes thread-agnostic, meaning it no longer distinguishes between different threads and only sees independent micro-operations.

Some pipeline stages must still maintain per-thread state. In particular, the Fetch, Decode, and Allocate (register renaming) stages are duplicated for each thread. Each thread has its own program counter and its own register mapping table, which ensures correct tracking of architectural registers during renaming. Retire (commit) stage must also know its thread identity, since it is responsible for committing instructions in order and freeing the correct physical registers associated with each thread.

There are two main strategies for sharing pipeline structures in SMT systems.

- The first is static partitioning, where each thread is assigned a fixed portion of the hardware resources.
- The other is dynamic sharing, it means all threads use the same hardware resources, with each entry tagged to indicate which thread it belongs to.

The implementation look then like this:



The critical decision for SMT is the fetch-interleaving policy: since multiple threads are active, the processor must choose which thread provide instructions to the pipeline at any given time.

The requirement to improve throughput is that we should have enough threads to utilize resources but fewer than needed to stretch dependences.

So how to decide which threads fetch instructions? The goal is to maximize active use of issue width.

ICOUNT is the common policy: threads with fewest instructions in pipe has priority. ICOUNT does not need to know why a thread is stalled, It just observes: “this thread has fewer active instructions in the pipeline → it is more stalled”.

SMT performs well when threads do not interfere with each other’s state in shared structures such as caches, branch predictors, and the ROB. It requires sufficient hardware resources to be effective; otherwise, extra thread context cannot be exploited.

It is especially well suited to wide-issue out-of-order processors, where unused execution slots can be filled by other threads.

Remark

Private caches for SMT are easier to implement but shared caches are faster to communicate among threads, there is no coherence overhead and offers more flexibility in allocating resources.

13 GPUs

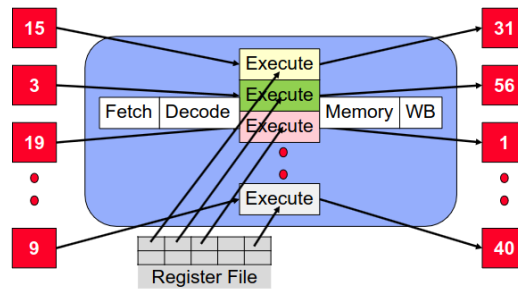
The idea of **GPUs** is to make very simple cores: in order pipeline, no branch prediction. Then, cores are smaller and we can put many cores in the die and take throughput-latency trade-off to extreme.

In GPUs there is two levels of multithreading:

- Vector lane threading: same as SIMD (Single Instruction Multiple Data), we group the thread in **warp**, and all threads in the warp execute the same instruction simultaneously on different data elements.
- Multithreading across pipeline depth: multiple warps can share the pipeline.

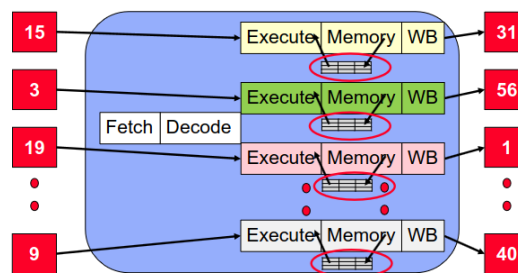
This is data level parallelism.

There is multiple way to compute identically different data items. We already saw the classic multi-core way (not optimal if it is only for identical instructions on every data) and the SIMD way, recall:



With SIMD, every lane operate on the same registers, it is specifically useful to operate on objects like vectors, but then there is no flexibility.

On GPUs, we apply have the **single instruction multiple thread (SIMT)** execution model. It seems similar as SIMD but each lane can run independent threads in “lockstep”. Threads operate on different registers (have its own separate context). SIMT is SIMD with hardware multithreading.



If we summary:

- MIMD/SPMD: this is good for thread level parallelism but bad for data level parallism. It’s the easiest programming way.
- SIMD/Vector: this is good for data level parallelism but the programmability is then limited and it’s hard to scale up.
- SIMT: this is good for data level parallelism, the programmability is better and it’s easy to scale up.

MIMD/SPMD



SIMD/Vector

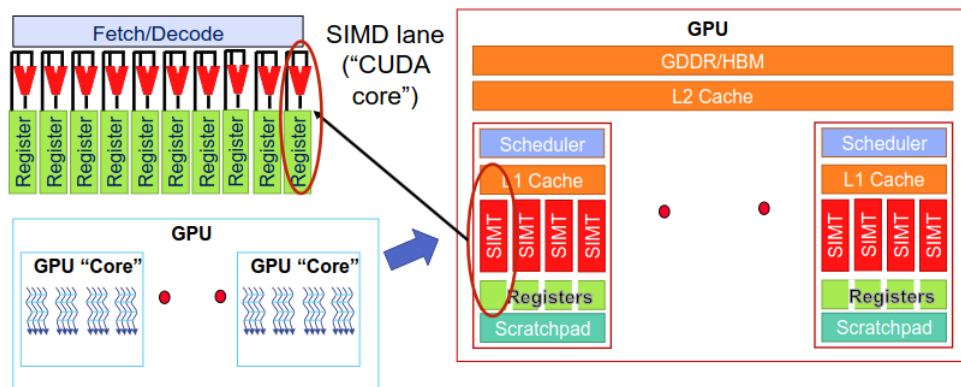


SIMT



13.1 GPU’s hardware

The GPU hardware is a set of GPU core (streaming multiprocessor (SM) for NVIDIA). There is around 10-100 SMs in a GPU and inside there is typically 4 SIMT. Inside SIMT there is generally 8 SIMD lane (“CUDA core”), so 32 in a SM.



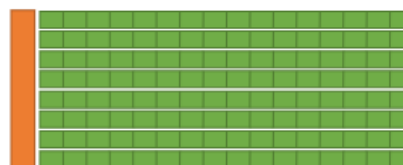
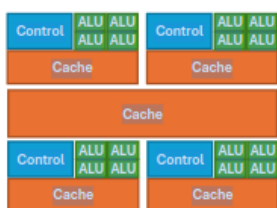
If we compare with the hardware of a CPU, we would see

for **CPU**:

- A few large ALUs,
- Large caches,
- Branch prediction, speculation and OoO,
- High clock frequency,
- Limited multithreading.

for **GPU**:

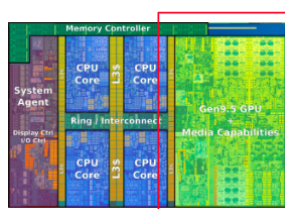
- A sea of small alus,
- Small, shallow cache hierarchy,
- Simple control logic, in-order,
- Massive multithreading,
- High memory bandwidth.



But GPU is a co-processor, it needs a companion CPU, this is the CPU that launch work and get back result.

Remark

We now have also little GPUs that are integrate on-chip (iGPU), this is a same architecture that a GPU but with much less resources. This is typically used for driving graphics in desktops/laptops.



14 CUDA

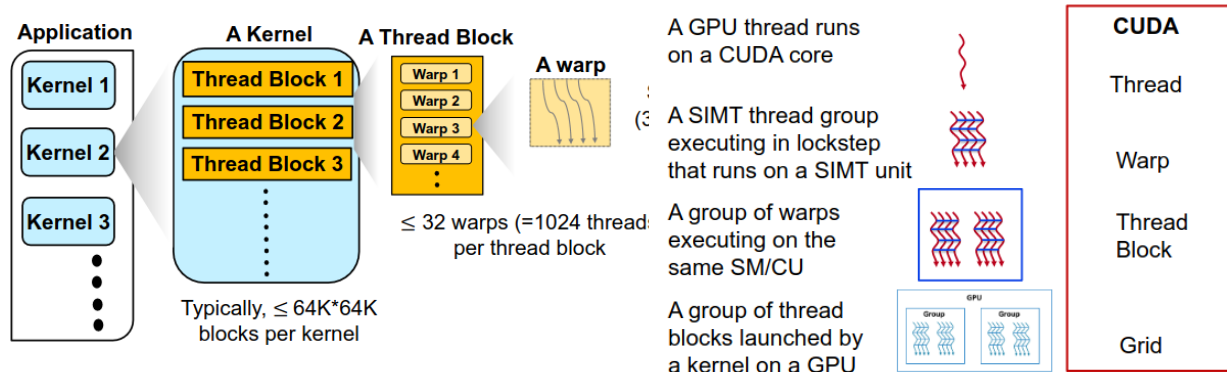
There is multiple GPU but we will focus on **CUDA (Compute Unified Device Architecture)**. This is the most widely used.

There is no “pure” GPU program, this is always a host or CPU program that launches work on GPU. It’s only accelerating functions (that are called GPU kernel).

GPU kernels follow the SIMT model:

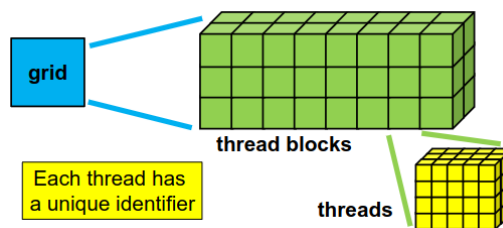
- GPU kernel code specifies what each thread must do,
- Thread index specifies which data the thread must operate on,
- Kernel is launched by the CPU, specifying how many threads should execute the kernel.

Let's just recall the organization hierarchy:

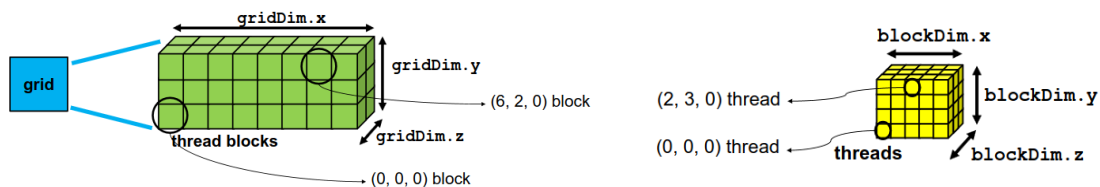


An advantage of breaking threads into thread blocks is that the CUDA runtime automatically distributes them according to the available GPU resources, and that threads within a thread block can collaborate more efficiently.

A kernel is launched with a grid of threads, a grid is organized as a 3D array of thread blocks and threads.



Each block and thread has a unique index tuple with three coordinates.



So the thread's global coordinates are:

```

1 x = blockIdx.x * blockDim.x + threadIdx.x
2 y = blockIdx.y * blockDim.y + threadIdx.y
3 z = blockIdx.z * blockDim.z + threadIdx.z

```